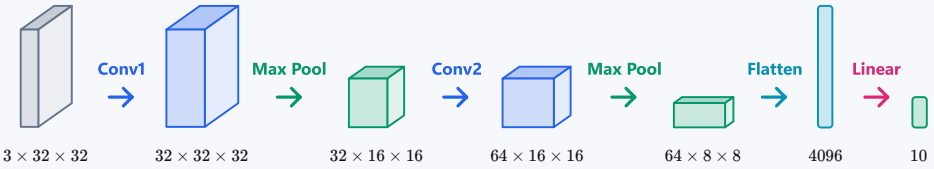
Bài giảng · Dành cho người bắt đầu từ số 0

GIÁO TRÌNH

Convolutional Neural Network — CNN

Từ một bức ảnh chỉ là ma trận số tới một mạng CNN huấn luyện được trên CIFAR-10: tính tay từng phép convolution (một channel, ảnh RGB, nhiều filter), stride và padding, công thức kích thước output, số tham số, ReLU, pooling, flatten, softmax và cross-entropy, backpropagation — kèm code PyTorch kiểm chứng từng bước, bảng theo dõi shape, những lỗi thường gặp và bài tập có lời giải.



VÍ DỤ CONVOLUTION XUYÊN SUỐT BÀI (MỤC 7–8)

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \star \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 37 & 47 \\ 67 & 77 \end{bmatrix}$$

18	104	424	39	55
phần	mục	công thức LaTeX	khối code Python	hình & sơ đồ

Mục lục

18 phần · 104 mục

Phần I **Bài toán và ảnh trong máy tính** 5

1 Mục tiêu của bài học 5
2 Bài toán đặt ra 6
3 Ảnh trong máy tính 7
3.1 Ảnh grayscale 7
4 Ảnh màu RGB 7
5 Tại sao không chỉ Flatten ảnh ngay từ đầu? 8

Phần II **Convolution trên ảnh một channel** 10

6 Ý tưởng trung tâm của CNN 10
7 Convolution trực quan 11
8 Kernel tiếp tục trượt 12
9 Convolution hay Cross-Correlation? 13
10 Feature Map 13
11 Kernel có thể học gì? 14
12 Kiểm chứng phép convolution bằng PyTorch 15

Phần III **Stride, padding và kích thước output** 16

13 Stride 16
13.1 Stride là gì? 16
14 Ví dụ stride = 2 16
15 Padding 17
16 Công thức tính chiều cao output 18
17 Trường hợp kernel vuông 18
18 Ví dụ tính kích thước output 18
19 Ví dụ stride 2 19

Phần IV **Convolution trên ảnh nhiều channel** 20

20 Ảnh màu và convolution 20
21 Ví dụ tính tay convolution RGB 20
22 Kiểm chứng ví dụ RGB bằng PyTorch 22

Phần V **Nhiều filter và số tham số của Conv2d** 23

23 Một filter tạo một output channel 23
24 Nhiều filter 23
25 Tensor weight của Conv2d 24
26 Số tham số của Conv2d 24
27 Ví dụ số tham số 25
28 Kiểm chứng bằng PyTorch 25
29 Weight Sharing 25
30 Local Connectivity 26

Phần VI **Activation: ReLU** 27

31 Activation Function 27
32 ReLU 27
33 Ví dụ ReLU 28
34 PyTorch ReLU 28
35 Conv + ReLU 29

Phần VII Pooling 30

36 Pooling 30
37 Max Pooling 30
38 Công thức Max Pooling 31
39 Average Pooling 31
40 PyTorch Max Pooling 32
41 Pooling có làm thay đổi số channel không? 32

Phần VIII Xếp chồng nhiều tầng 34

42 Một CNN block điển hình 34
43 CNN nhiều tầng 35
44 Tại sao channel thường tăng? 35
45 Receptive Field trực quan 36

Phần IX Từ feature map tới xác suất và loss 37

46 Flatten 37
47 PyTorch Flatten 38
48 Fully Connected Layer 38
49 Logits 39
50 Softmax 39
51 Ví dụ Softmax 39
52 Cross-Entropy Loss 40
53 Ví dụ Cross-Entropy 41
54 Không cần gọi Softmax trước CrossEntropyLoss trong PyTorch 41

Phần X CNN hoàn chỉnh 42

55 Kiến trúc CNN hoàn chỉnh 42
56 Theo dõi tensor bằng một ví dụ hoàn chỉnh 42
57 CNN hoàn chỉnh bằng PyTorch 43
58 Ý nghĩa batch dimension 45
59 `nn.Conv2d` 45
60 Kiểm tra weight 46

Phần XI CNN học như thế nào 47

61 CNN học filter như thế nào? 47
62 Forward Pass 48
63 Backpropagation 48
64 Cập nhật bias 49
65 Kernel không được thiết kế thủ công trong CNN hiện đại 49
66 Optimizer 50
67 Một bước training hoàn chỉnh 50
68 Chương trình training CNN hoàn chỉnh trên CIFAR-10 50
69 Giải thích `argmax` 53

Phần XII Thành phần của CNN hiện đại 55

70 Global Average Pooling 55
71 Ví dụ Global Average Pooling 56
72 PyTorch Global Average Pooling 56
73 Batch Normalization 57
74 Dropout 57
75 Data Augmentation 58

Phần XIII Huấn luyện và đánh giá 59

76 Train mode và eval mode 59
77 Validation 59
78 Overfitting 60
79 Underfitting 60

Phần XIV Hiểu sâu CNN 61

80 CNN sâu hơn 61
81 CNN học đặc trưng theo cấp độ 61
82 Convolution là phép Linear Transformation cục bộ 62
83 Ví dụ khai triển một patch RGB 62
84 Hai ý tưởng cốt lõi làm CNN hiệu quả 63
85 Ví dụ so sánh số tham số 63
86 Một CNN tốt không nhất thiết cần Flatten lớn 63

Phần XV Kiến trúc nổi tiếng và Residual Connection 65

87 Các kiến trúc CNN quan trọng trong lịch sử Deep Learning 65
88 Residual Connection 65
89 Một Residual Block đơn giản 66
90 Tại sao hai tensor cộng được? 68
91 CNN classification pipeline đầy đủ 68
92 CNN không chỉ dùng cho classification 68

Phần XVI Công thức, shape và lỗi thường gặp 69

93 Những công thức phải thuộc và hiểu 69
94 Bảng theo dõi shape cần biết đọc 69
95 Chương trình theo dõi shape tự động 70
96 Những lỗi người mới thường gặp 72

Phần XVII Bài tập 74

97 Bài tập 1 — tính output shape 74
98 Bài tập 2 — tính output shape với stride 2 74
99 Bài tập 3 — tính số parameter 75
100 Bài tập 4 — theo dõi toàn bộ network 75

Phần XVIII Tổng kết 76

101 Bản chất toàn bộ CNN trong một chuỗi công thức 76
102 Mô hình tư duy quan trọng nhất 77
103 Chuỗi tư duy từ pixel tới classification 77
104 Kết luận kiến thức cốt lõi 78

I

PHẦN I

Mục 1–5 · 5 mục

Bài toán và ảnh trong máy tính

Mục tiêu của bài học và bài toán phân loại ảnh; ảnh grayscale là ma trận $H \times W$, ảnh RGB là tensor $C \times H \times W$; vì sao Flatten ảnh ngay từ đầu vừa tốn tham số vừa làm mất cấu trúc không gian.

1 Mục tiêu của bài học

2 Bài toán đặt ra

3 Ảnh trong máy tính

4 Ảnh màu RGB

5 Tại sao không chỉ Flatten ảnh ngay từ đầu?

Mục 1 Mục tiêu của bài học

Phần I

Sau khi học xong bài này, người học cần hiểu được:

1	Ảnh được biểu diễn trong máy tính như thế nào.	Mục 3–4	2	CNN là gì và tại sao cần CNN.	Mục 5–6
3	Convolution là gì.	Mục 7–9	4	Kernel, filter, feature map, channel là gì.	Mục 10
5	Cách kernel trượt trên ảnh.	Mục 8	6	Cách tính từng phần tử của feature map bằng tay.	Mục 7–8
7	Cách convolution hoạt động với ảnh RGB.	Mục 20–22	8	Cách nhiều filter tạo ra nhiều output channel.	Mục 23–25
9	Vai trò của stride.	Mục 13–14	10	Vai trò của padding.	Mục 15
11	Công thức tính kích thước output.	Mục 16–19	12	Bias trong convolution.	Mục 21
13	Activation function.	Mục 31	14	ReLU.	Mục 32–34
15	Pooling.	Mục 36	16	Max Pooling.	Mục 37–40
17	Average Pooling.	Mục 39	18	Flatten.	Mục 46–47
19	Fully Connected Layer.	Mục 48	20	Logits.	Mục 49
21	Softmax.	Mục 50–51	22	Cross-Entropy Loss.	Mục 52–54
23	CNN học kernel như thế nào.	Mục 61–65	24	Backpropagation trong CNN ở mức cần thiết.	Mục 63–64
25	Batch và tensor 4 chiều trong PyTorch.	Mục 58	26	`nn.Conv2d`.	Mục 59–60
27	`nn.MaxPool2d`.	Mục 40	28	Cách xây dựng một CNN hoàn chỉnh.	Mục 55–57

29	Quy trình training CNN.	Mục 61–68	
30	Cách theo dõi kích thước tensor qua toàn bộ mạng.	Mục 56	

Nhãn "Mục ..." bên phải mỗi mục tiêu dẫn tới phần bài giảng trả lời mục tiêu đó.

Mục tiêu cuối cùng là khi nhìn vào một kiến trúc như:

$$\text{Image} \to \text{Conv} \to \text{ReLU} \to \text{Pool} \to \text{Conv} \to \text{ReLU} \to \text{Pool} \to \text{Classifier}$$

người học có thể hiểu chính xác chuyện gì đang xảy ra ở từng bước.

Mục 2 Bài toán đặt ra

Phần I

Giả sử cần xây dựng mô hình nhận một bức ảnh và xác định ảnh đó thuộc lớp nào.

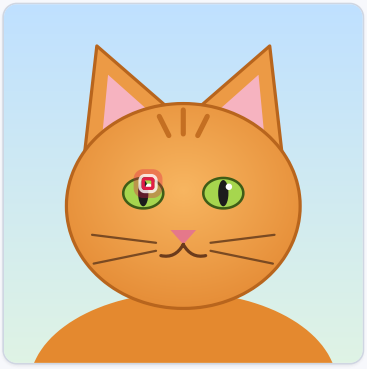
Ví dụ:

$$\text{ảnh} \to \begin{cases} \text{cat} \\ \text{dog} \\ \text{bird} \end{cases}$$

MÁY TÍNH KHÔNG TRỰC TIẾP NHÌN THẤY

Đây là con mèo.

Máy tính chỉ nhận một tập hợp các số.



con người nhìn thấy

phóng to →

76	76	76	76	82	109	141	179
133	133	137	130	109	83	76	81
28	28	120	184	184	184	148	105
28	28	65	167	184	184	184	184
28	28	28	214	255	234	184	184
28	28	127	255	255	255	215	184
28	28	127	255	255	255	215	184
28	28	28	186	255	234	184	184

máy tính nhận 8×8 số (ảnh xám)

HÌNH 2.1 Cùng một vùng ảnh dưới hai góc nhìn. Khối 8×8 pixel được tô viền (quanh mắt trái) đổi sang độ xám 0–255: 28 là con ngươi gần đen, 255 là đốm sáng trắng, 76 là đường viền mắt, 184 là tròng mắt. Các số được đo trực tiếp trên hình vẽ.

Do đó, bài toán thực chất là:

$$\text{ma trận số} \to \text{mô hình} \to \text{vector điểm số} \to \text{nhãn}$$

CNN là một kiến trúc Neural Network được thiết kế đặc biệt để xử lý dữ liệu có cấu trúc không gian như ảnh.

Mục 3 Ảnh trong máy tính

Phần I

3.1 Ảnh grayscale

Một ảnh grayscale có thể được xem như một ma trận:

$$X \in \mathbb{R}^{H \times W}$$

Trong đó:

- H chiều cao ảnh;
- W chiều rộng ảnh.

Ví dụ:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$



HÌNH 3.1 Mỗi phần tử của X là độ sáng của một pixel: $H = 3$ hàng, $W = 3$ cột. Ở đây số lớn được vẽ sáng hơn (cách tô chỉ để minh họa).

Đây là ảnh $H = 3,\qquad W = 3$

Mục 4 Ảnh màu RGB

Phần I

Ảnh RGB gồm ba channel:

R **Red;** G **Green;** B **Blue.**

Do đó:

$$X \in \mathbb{R}^{C \times H \times W}$$

với $C = 3$ cho ảnh RGB.

Ví dụ:

$$X \in \mathbb{R}^{3 \times 32 \times 32}$$

có nghĩa là:

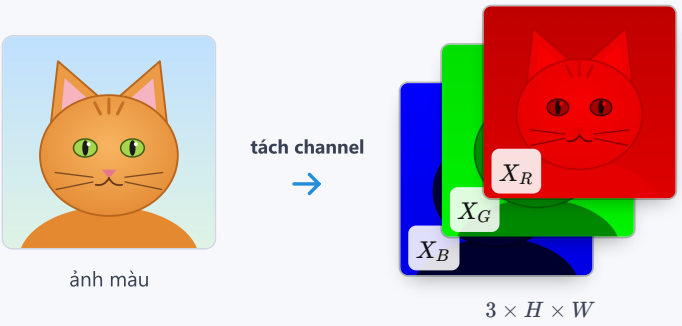
$$X \left[\boxed{3}, \boxed{32}, \boxed{32} \right]$$

3 channel · cao 32 pixel · rộng 32 pixel

Ta có thể tưởng tượng:

$$X = \begin{bmatrix} X_R \\ X_G \\ X_B \end{bmatrix}$$

trong đó mỗi X_R, X_G, X_B là một ma trận $H \times W$.



ảnh màu

$3 \times H \times W$

HÌNH 4.1 Mỗi channel giữ một thành phần màu của mọi pixel: X_R (đỏ), X_G (xanh lá), X_B (xanh dương), mỗi cái là ma trận $H \times W$. Xếp chồng ba ma trận được tensor $C \times H \times W$ với $C = 3$.

Mục 5 Tại sao không chỉ Flatten ảnh ngay từ đầu?

Phần I

1. **Giả sử ảnh RGB có kích thước** 224×224
2. **Số giá trị đầu vào là** $3 \times 224 \times 224 = 150528$
3. **Nếu nối trực tiếp với một Dense Layer có 1000 neuron**
   $150528 \times 1000 = 150\,528\,000$
   weight.

⚠ **Đây là số lượng tham số rất lớn.**

☆ **Ngoài ra Flatten còn làm mất cấu trúc không gian.**



HÌNH 5.1 Các ô được đánh số theo thứ tự flatten (đọc từng hàng). Bốn pixel kề nhau $6, 7, 10, 11$ tạo thành một vùng nhỏ của ảnh, nhưng trong vector chúng bị tách thành hai cặp; Dense Layer không còn biết pixel nào nằm cạnh pixel nào.

Trong ảnh, pixel gần nhau thường có quan hệ với nhau.

Ví dụ:

cạnh **góc** **đường cong** **texture** **bộ phận của vật thể**

CNN tận dụng cấu trúc này.

II

PHẦN II

Mục 6–12 · 7 mục

Convolution trên ảnh một channel

Kernel trượt trên ảnh: tính tay từng phần tử của feature map với ảnh 3×3 và kernel 2×2; convolution hay cross-correlation; kernel học được gì; kiểm chứng bằng PyTorch.

6 Ý tưởng trung tâm của CNN

7 Convolution trực quan

8 Kernel tiếp tục trượt

9 Convolution hay Cross-Correlation?

10 Feature Map

11 Kernel có thể học gì?

12 Kiểm chứng phép convolution bằng PyTorch

Mục 6 Ý tưởng trung tâm của CNN

Phần II

Thay vì mỗi neuron kết nối với toàn bộ ảnh, CNN sử dụng một cửa sổ nhỏ gọi là **kernel** hoặc **filter**.

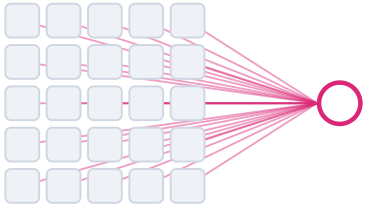
Kernel lần lượt quan sát các vùng nhỏ của ảnh.

Ví dụ kernel:

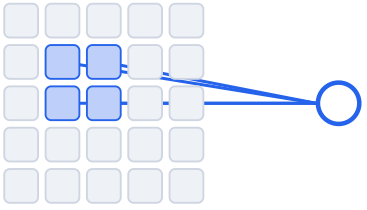
$$K = \begin{bmatrix} k_{11} & k_{12} \\ k_{21} & k_{22} \end{bmatrix}$$

Kernel này có kích thước 2×2

Nó sẽ lần lượt xử lý các vùng 2×2 của ảnh.



Dense: một neuron nối với cả 25 pixel



Kernel 2×2: chỉ nhìn 4 pixel của một vùng nhỏ

HÌNH 6.1 Trên ảnh 5×5: bên trái, mỗi neuron của Dense Layer cần một weight cho từng pixel; bên phải, kernel 2×2 chỉ nhìn 4 pixel tại một vị trí rồi trượt sang vùng khác.

Cho

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

và kernel

$$K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

1. **Đặt kernel tại góc trên bên trái** $\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
2. **Sau đó nhân từng phần tử tương ứng** $1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4$
3. **Ta có**
$$= 1 + 4 + 12 + 20$$
$$= 37$$
4. **Vì vậy** $Y_{1,1} = 37$

Đây chính là một phần tử của output.



HÌNH 7.1 Vùng tô xanh của X nằm dưới kernel. Mỗi ô của vùng nhân với ô cùng vị trí của K (1×1, 2×2, 4×3, 5×4); cộng bốn tích được $Y_{1,1} = 37$. Ba ô còn lại của Y được tính ở Mục 8.

Kernel dịch sang phải một pixel. Vùng tiếp theo

$$\begin{bmatrix} 2 & 3 \\ 5 & 6 \end{bmatrix}$$

Ta tính

$$\begin{aligned} Y_{1,2} &= 2 \times 1 + 3 \times 2 + 5 \times 3 + 6 \times 4 \\ &= 2 + 6 + 15 + 24 \\ &= 47 \end{aligned}$$

Tiếp theo kernel xuống hàng dưới. Vùng

$$\begin{bmatrix} 4 & 5 \\ 7 & 8 \end{bmatrix}$$

$$\begin{aligned} Y_{2,1} &= 4 \times 1 + 5 \times 2 + 7 \times 3 + 8 \times 4 \\ &= 4 + 10 + 21 + 32 \\ &= 67 \end{aligned}$$

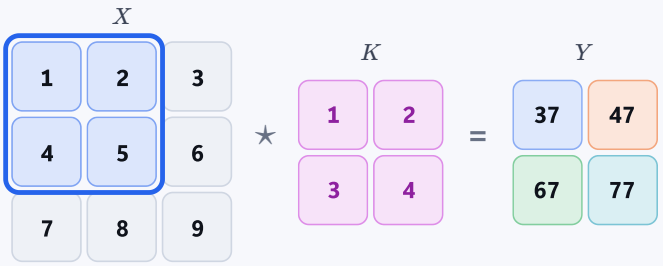
Vùng cuối

$$\begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$$

$$\begin{aligned} Y_{2,2} &= 5 \times 1 + 6 \times 2 + 8 \times 3 + 9 \times 4 \\ &= 5 + 12 + 24 + 36 \\ &= 77 \end{aligned}$$

Output hoàn chỉnh:

$$Y = \begin{bmatrix} 37 & 47 \\ 67 & 77 \end{bmatrix}$$



$$\begin{aligned} Y_{1,1} &= 1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4 \\ &= 1 + 4 + 12 + 20 = 37 \end{aligned}$$

$$\begin{aligned} Y_{1,2} &= 2 \times 1 + 3 \times 2 + 5 \times 3 + 6 \times 4 \\ &= 2 + 6 + 15 + 24 = 47 \end{aligned}$$

$$\begin{aligned} Y_{2,1} &= 4 \times 1 + 5 \times 2 + 7 \times 3 + 8 \times 4 \\ &= 4 + 10 + 21 + 32 = 67 \end{aligned}$$

$$\begin{aligned} Y_{2,2} &= 5 \times 1 + 6 \times 2 + 8 \times 3 + 9 \times 4 \\ &= 5 + 12 + 24 + 36 = 77 \end{aligned}$$

HÌNH 8.1 Bốn phép tính ứng với bốn vị trí của kernel (khung trên X là vị trí đầu tiên; bản HTML cho phép bấm để trượt kernel). Mỗi màu là một vị trí của kernel và phần tử Y mà nó tạo ra; ký hiệu $\star$ chỉ phép convolution. Kernel 2×2 vừa đúng $2 \times 2 = 4$ vị trí trên ảnh 3×3, nên Y có kích thước 2×2.

Mục 9 Convolution hay Cross-Correlation?

Phần II

Toán học

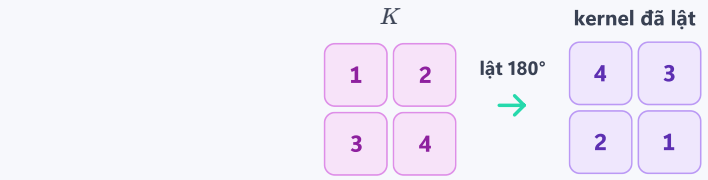
Trong toán học, convolution truyền thống thường đảo kernel trước khi tính.

Deep Learning · `Conv2d`

Trong Deep Learning, thao tác được các thư viện như PyTorch gọi là `Conv2d` thực tế sử dụng phép nhân trực tiếp kernel với vùng input mà không cần đảo kernel.

Trong học sâu, quy ước này được gọi phổ biến là convolution và không gây vấn đề vì các giá trị kernel được mô hình tự học.

Trong giáo trình này, từ convolution được sử dụng theo quy ước thông dụng trong Deep Learning.



Conv2d (không lật)

$$1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4 = 37$$

convolution toán học (lật kernel)

$$1 \times 4 + 2 \times 3 + 4 \times 2 + 5 \times 1 = 23$$

HÌNH 9.1 Ví dụ của Mục 7 theo hai quy ước (bài giảng bổ sung): convolution trong toán học lật kernel $180°$ (đảo cả hàng lẫn cột) trước khi nhân, nên cùng vị trí cho 23 thay vì 37. Vì kernel được học, mô hình chỉ cần học phiên bản đã lật, nên hai cách mạnh như nhau.

Mục 10 Feature Map

Phần II

Kết quả của một filter được gọi là: feature map.

Ví dụ:

$$X \xrightarrow{K} Y$$

Trong đó:

- X ảnh đầu vào;
- K filter;
- Y feature map.

Mỗi feature map biểu diễn mức độ mà một đặc trưng nào đó xuất hiện tại từng vùng của ảnh.

Một số kernel có thể phản ứng mạnh với:

cạnh dọc **cạnh ngang** **góc** **đường cong** **texture**



HÌNH 11.1 Hai kernel 3×3 đặt tay để minh họa, áp dụng bằng `F.conv2d` lên ảnh xám. Chỗ càng đậm, kernel phản ứng càng mạnh: kernel thứ nhất (lấy cột trái trừ cột phải) bắt các cạnh dọc như hai bên má, các sọc trên trán; kernel thứ hai (chuyển vị của nó: lấy hàng trên trừ hàng dưới) bắt các cạnh ngang như ria mèo. Trong CNN, các kernel như vậy được học từ dữ liệu (Mục 65).

Trong mạng sâu hơn, các layer có thể dần học:

pixel → edge → texture → part → object

Ví dụ:

pixel → đường biên → mắt → khuôn mặt

QUAN TRỌNG

Các filter không cần do con người thiết kế thủ công.

CNN học chúng thông qua training.

PYTHON 27 dòng

```python
import torch
import torch.nn.functional as F

x = torch.tensor([
    [1., 2., 3.],
    [4., 5., 6.],
    [7., 8., 9.]
])

kernel = torch.tensor([
    [1., 2.],
    [3., 4.]
])

# PyTorch Conv2d cần dạng:
# [batch, channel, height, width]

x = x.unsqueeze(0).unsqueeze(0)

# Weight cần dạng:
# [out_channels, in_channels, kernel_h, kernel_w]

kernel = kernel.unsqueeze(0).unsqueeze(0)

y = F.conv2d(x, kernel)

print(y)
```

KẾT QUẢ

```
tensor([[[[37., 47.],
          [67., 77.]]]])
```

✓ Đúng với kết quả tính tay.

III

PHẦN III

Mục 13–19 · 7 mục

Stride, padding và kích thước output

Stride là bước trượt của kernel, padding là viền thêm quanh ảnh; công thức $H_{\text{out}} = \lfloor (H_{\text{in}} + 2P - K)/S \rfloor + 1$ và hai ví dụ tính kích thước output.

13 Stride

14 Ví dụ stride = 2

15 Padding

16 Công thức tính chiều cao output

17 Trường hợp kernel vuông

18 Ví dụ tính kích thước output

19 Ví dụ stride 2

Mục 13 Stride

Phần III

13.1 Stride là gì?

Stride xác định số pixel kernel dịch chuyển sau mỗi bước.

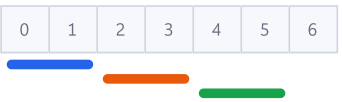
Nếu $S = 1$

kernel dịch 1 pixel mỗi lần.



Nếu $S = 2$

kernel dịch 2 pixel mỗi lần.



Mục 14 Ví dụ stride = 2

Phần III

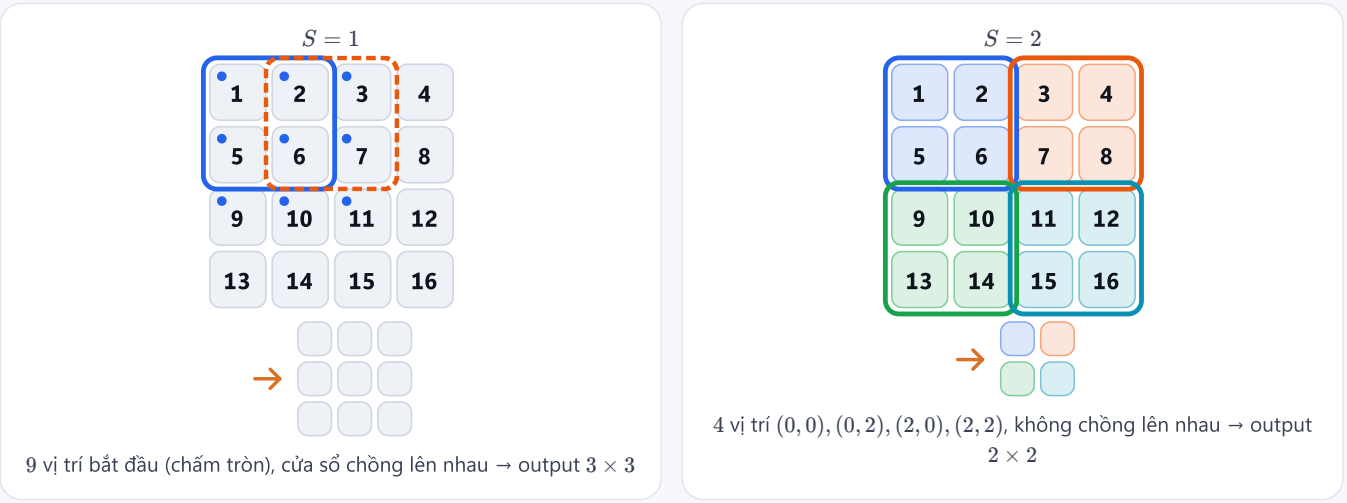
Input:

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Kernel 2×2.

Với stride $S = 2$

kernel lần lượt bắt đầu tại $(0,0), (0,2), (2,0), (2,2)$ thay vì đi qua mọi vị trí.



HÌNH 14.1 Kernel 2×2 trên input 4×4 của Mục 14. Với $S = 1$, cửa sổ thứ hai (nét đứt) chỉ dịch một cột nên chồng lên cửa sổ đầu; với $S = 2$, bốn cửa sổ vừa khít, output nhỏ đi còn 2×2.

Stride lớn hơn thường làm kích thước không gian của output nhỏ hơn.

Mục 15 Padding

Phần III

Nếu không thêm phần tử xung quanh ảnh, feature map thường nhỏ hơn input.

Ví dụ 5×5 convolution với kernel 3×3 stride 1 tạo output 3×3

Để kiểm soát sự giảm kích thước, ta có thể thêm một biên xung quanh ảnh.





HÌNH 15.1 Viền số 0 (ô gạch chéo) cho phép kernel 3×3 đặt tâm ngay tại pixel góc 1. Tâm kernel đi qua đủ 9 pixel gốc, nên output giữ nguyên 3×3; không có padding thì chỉ còn 1×1.

Mục 16 Công thức tính chiều cao output

Phần III

Giả sử:

H_{in}	K_H	P_H	S_H
input height	kernel height	padding	stride

Khi đó:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_H - K_H}{S_H} \right\rfloor + 1$$

Tương tự:

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_W - K_W}{S_W} \right\rfloor + 1$$

Mục 17 Trường hợp kernel vuông

Phần III

Nếu:

$$K_H = K_W = K \qquad P_H = P_W = P \qquad S_H = S_W = S$$

thì:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1 \quad \text{và} \quad W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

Mục 18 Ví dụ tính kích thước output

Phần III

Input	Kernel	Padding	Stride
32×32	3×3	$P = 1$	$S = 1$

1. **Ta có**
$$H_{\text{out}} = \frac{32 + 2(1) - 3}{1} + 1 = 32$$
2. **Tương tự** $W_{\text{out}} = 32$
3. **Vì vậy** $32 \times 32 \to 32 \times 32$

Input	Kernel	Padding	Stride
32×32	3×3	1	2

1 **Ta có**

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{32 + 2 - 3}{2} \right\rfloor + 1 \\ &= \lfloor 15.5 \rfloor + 1 \\ &= 16 \end{aligned}$$

2 **Do đó** $32 \times 32 \to 16 \times 16$

H_{in} **32** K **3** P **1** S **2**

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{32 + 2 \cdot 1 - 3}{2} \right\rfloor + 1 \\ &= \lfloor 15.5 \rfloor + 1 = \mathbf{16} \end{aligned}$$



Kernel đặt được **16** vị trí trên mỗi chiều; 1 ô cuối cùng không được kernel nào phủ tới. Với ảnh vuông, output là **16 × 16**.

HÌNH 19.1 Bản in cho thấy đúng ví dụ ở trên ($H_{\text{in}} = 32$, $K = 3$, $P = 1$, $S = 2$); trong bản HTML có thể đổi các giá trị để thấy công thức của Mục 16–17 thay số. Hàng ô bên dưới là một hàng của input đã thêm padding (ô gạch chéo là padding); mỗi vạch màu là một vị trí đặt kernel, số vị trí đúng bằng H_{out}. Công cụ này là phần bổ sung của bài giảng.

IV

PHẦN IV

Mục 20–22 · 3 mục

Convolution trên ảnh nhiều channel

Mỗi filter có độ sâu bằng số input channel; tính tay một vùng RGB 2×2 (12 phép nhân) cho $Y = 53$ và kiểm chứng bằng PyTorch.

20 Ảnh màu và convolution

21 Ví dụ tính tay convolution RGB

22 Kiểm chứng ví dụ RGB bằng PyTorch

Mục 20 Ảnh màu và convolution

Phần IV

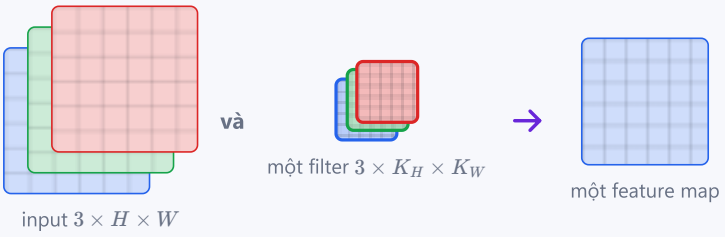
☆ **Đây là điểm rất quan trọng.**

Nếu input có $C_{\text{in}} = 3$ thì mỗi filter cũng phải có độ sâu bằng 3.

Ví dụ
$X \in \mathbb{R}^{3 \times H \times W}$

thì một filter
$K \in \mathbb{R}^{3 \times K_H \times K_W}$

ⓘ **Không phải một ma trận $K_H \times K_W$, mà gồm ba kernel tương ứng ba input channel.**



HÌNH 20.1 Filter có đúng 3 lớp, mỗi lớp ứng với một channel của input (cùng màu). Filter chỉ trượt theo chiều cao và chiều rộng; ba lớp được tính cùng lúc rồi cộng lại, nên kết quả chỉ là một feature map (ví dụ tính tay ở Mục 21).

Mục 21 Ví dụ tính tay convolution RGB

Phần IV

Xét một vùng RGB kích thước 2×2

R Red channel

$$X_R = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

G Green channel

$$X_G = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$$

B Blue channel

$$X_B = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}$$

Filter có:

$$K_R = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \qquad K_G = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \qquad K_B = \begin{bmatrix} 1 & 3 \\ 2 & 2 \end{bmatrix}$$

và $b = 1$

Ta tính riêng từng channel.

R Red

$$1(1) + 2(2) + 3(2) + 4(1) = 1 + 4 + 6 + 4 = 15$$

G Green

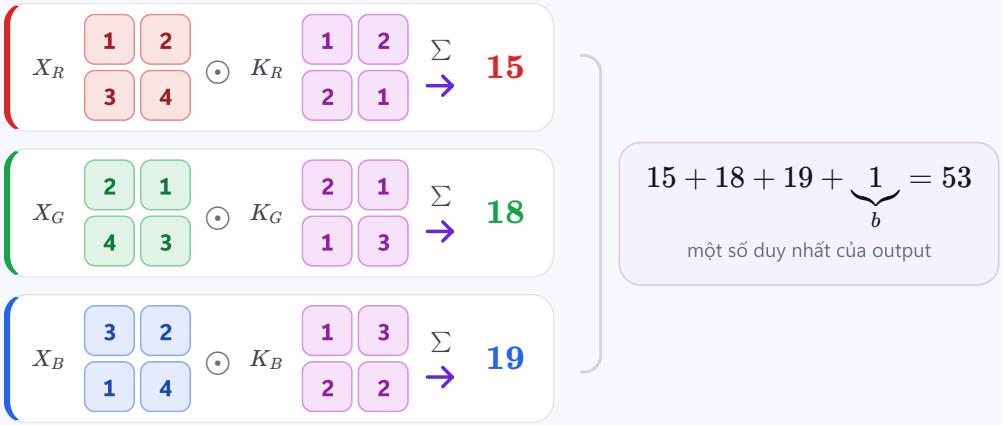
$$2(2) + 1(1) + 4(1) + 3(3) = 4 + 1 + 4 + 9 = 18$$

B Blue

$$3(1) + 2(3) + 1(2) + 4(2) = 3 + 6 + 2 + 8 = 19$$

- **Cộng ba channel và bias** $Y = 15 + 18 + 19 + 1$ $\quad Y = 53$

Tổng cộng $3 \times 2 \times 2 = 12$ phép nhân cho một output pixel.



HÌNH 21.1 Mỗi channel được nhân từng phần tử ($\odot$) với kernel của chính nó rồi cộng lại: 15, 18, 19. Ba tổng riêng cộng với bias $b = 1$ cho $Y = 53$: một filter xử lý cả ba channel nhưng chỉ tạo ra một output.

Điểm rất quan trọng:

> một filter xử lý tất cả input channel và tạo ra một output channel

PYTHON 42 dòng

```python
import torch
import torch.nn.functional as F

x = torch.tensor([
    [
        [1., 2.],
        [3., 4.]
    ],
    [
        [2., 1.],
        [4., 3.]
    ],
    [
        [3., 2.],
        [1., 4.]
    ]
])

weight = torch.tensor([
    [
        [
            [1., 2.],
            [2., 1.]
        ],
        [
            [2., 1.],
            [1., 3.]
        ],
        [
            [1., 3.],
            [2., 2.]
        ]
    ]
])

x = x.unsqueeze(0)

bias = torch.tensor([1.])

y = F.conv2d(x, weight, bias=bias)

print(y)
```

KẾT QUẢ

```
tensor([[[[53.]]]])
```

V

PHẦN V

Mục 23–30 · 8 mục

Nhiều filter và số tham số của Conv2d

Một filter tạo một output channel; tensor weight $[C_{\text{out}}, C_{\text{in}}, K_H, K_W]$; số tham số $C_{\text{out}}(C_{\text{in}}K_HK_W + 1)$; weight sharing và local connectivity.

23 Một filter tạo một output channel

24 Nhiều filter

25 Tensor weight của Conv2d

26 Số tham số của Conv2d

27 Ví dụ số tham số

28 Kiểm chứng bằng PyTorch

29 Weight Sharing

30 Local Connectivity

Mục 23 Một filter tạo một output channel

Phần V

Giả sử input $X \in \mathbb{R}^{3\times32\times32}$ và ta sử dụng một filter.

Output sẽ có 1 channel.

Nếu spatial size không đổi $Y \in \mathbb{R}^{1\times32\times32}$

Mục 24 Nhiều filter

Phần V

Trong thực tế, CNN cần phát hiện nhiều loại đặc trưng.

Ví dụ:

- filter 1 học edge;
- filter 2 học texture;
- filter 3 học curve;
- ...

Nếu sử dụng $C_{\text{out}} = 64$ filter thì output có 64 channel.

Do đó:

$$\text{số filter} = \text{số output channel}$$



HÌNH 24.1 Mỗi filter (cả 3 lớp) quét toàn bộ input và cho một feature map Y_o. 64 feature map xếp chồng thành 64 channel của cùng một output (giả sử spatial size không đổi như ở Mục 23).

Mục 25 Tensor weight của Conv2d

Phần V

Nếu $C_{\text{in}} = 3 \quad C_{\text{out}} = 64$ kernel 3×3

thì tensor weight có kích thước:

$$64 \times 3 \times 3 \times 3$$

Theo thứ tự:

$$[\ 64\ ,\ 3\ ,\ 3\ ,\ 3\ ]$$

C_{out} C_{in} K_H K_W

Mục 26 Số tham số của Conv2d

Phần V

1. **Mỗi filter có** $C_{\text{in}} \times K_H \times K_W$ weight.
2. **Có** C_{out} filter.
3. **Do đó số weight** $C_{\text{out}} C_{\text{in}} K_H K_W$
4. **Nếu mỗi output channel có một bias** C_{out} bias.
5. **Tổng**

$$N_{\text{params}} = C_{\text{out}} C_{\text{in}} K_H K_W + C_{\text{out}}$$

hay

$$N_{\text{params}} = C_{\text{out}} (C_{\text{in}} K_H K_W + 1)$$

Mục 27 Ví dụ số tham số

Phần V

Layer:

PYTHON 5 dòng

```python
nn.Conv2d(
    in_channels=3,
    out_channels=64,
    kernel_size=3
)
```

1	Số weight	$64 \times 3 \times 3 \times 3 = 1728$
2	Số bias	64
3	Tổng	$1728 + 64$ $\quad$ 1792

Mục 28 Kiểm chứng bằng PyTorch

Phần V

PYTHON 11 dòng

```python
import torch.nn as nn

conv = nn.Conv2d(
    in_channels=3,
    out_channels=64,
    kernel_size=3
)

total_params = sum(p.numel() for p in conv.parameters())

print(total_params)
```

KẾT QUẢ

```
1792
```

Mục 29 Weight Sharing

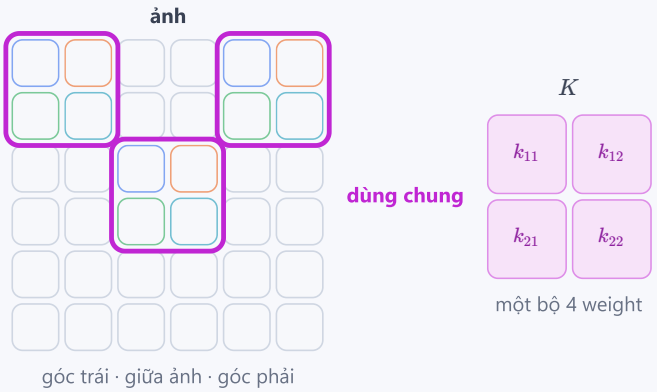
Phần V

Một đặc điểm rất quan trọng của CNN là cùng một kernel được sử dụng tại nhiều vị trí.

Ví dụ kernel K được dùng ở góc trái, giữa ảnh, góc phải,...

CNN không cần một bộ weight riêng cho mỗi vị trí.

Đây gọi là weight sharing



HÌNH 29.1 Ba vị trí khác nhau của kernel dùng cùng bốn weight $k_{11}, k_{12}, k_{21}, k_{22}$ (cùng màu); ô cùng màu nhân với cùng một weight. Số tham số của kernel không phụ thuộc vào kích thước ảnh.

Lợi ích:

- giảm mạnh số tham số;
- cùng một đặc trưng có thể được nhận diện ở nhiều vị trí;
- tận dụng cấu trúc ảnh.

Mục 30 Local Connectivity

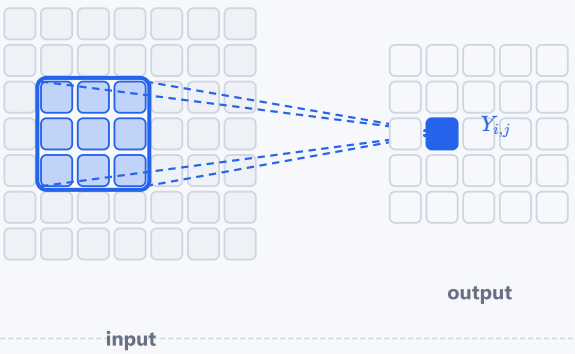
Phần V

Một output neuron không nhìn toàn bộ ảnh.

Nó chỉ nhìn một vùng nhỏ.

Ví dụ kernel 3×3 $Y_{i,j}$ chỉ phụ thuộc vào vùng 3×3 tương ứng của input.

Đây gọi là local connectivity



HÌNH 30.1 Với kernel 3×3 (stride 1, không padding), input 7×7 cho output 5×5. Phần tử $Y_{i,j}$ chỉ phụ thuộc 9 pixel trong khung; 40 pixel còn lại không ảnh hưởng tới nó.

VI

PHẦN VI

Mục 31–35 · 5 mục

Activation: ReLU

Vì sao cần hàm phi tuyến sau convolution; $\mathrm{ReLU}(x) = \max(0, x)$, ví dụ tính tay, PyTorch và block Conv + ReLU.

31 Activation Function

32 ReLU

33 Ví dụ ReLU

34 PyTorch ReLU

35 Conv + ReLU

Mục 31 Activation Function

Phần VI

Sau convolution, nếu chỉ liên tục thực hiện các phép biến đổi tuyến tính thì khả năng biểu diễn của mạng bị hạn chế.

☆ **Ta cần nonlinear activation.**

Một activation phổ biến là ReLU.

Mục 32 ReLU

Phần VI

ReLU được định nghĩa:

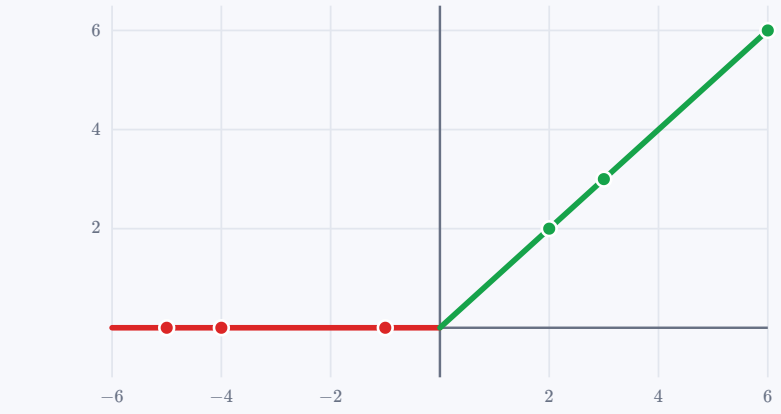
$$\mathrm{ReLU}(x) = \max(0, x)$$

Nếu $x > 0$

thì $\mathrm{ReLU}(x) = x$

Nếu $x \leq 0$

thì $\mathrm{ReLU}(x) = 0$



HÌNH 32.1 $\mathrm{ReLU}(x) = \max(0, x)$: nửa trái nằm trên trục hoành (mọi giá trị âm thành 0), nửa phải là đường $y = x$. Các chấm là sáu giá trị của ma trận ở Mục 33: $-4, -1, -5$ thành 0; $2, 3, 6$ giữ nguyên.

Mục 33 Ví dụ ReLU

Phần VI

CHO

$$X = \begin{bmatrix} -4 & 2 & -1 \\ 3 & -5 & 6 \end{bmatrix}$$

→

SAU RELU

$$Y = \begin{bmatrix} 0 & 2 & 0 \\ 3 & 0 & 6 \end{bmatrix}$$

Mục 34 PyTorch ReLU

Phần VI

PYTHON 11 dòng

```python
import torch
import torch.nn.functional as F

x = torch.tensor([
    [-4., 2., -1.],
    [3., -5., 6.]
])

y = F.relu(x)

print(y)
```

KẾT QUẢ KHI CHẠY (BÀI GIẢNG BỔ SUNG)

```
tensor([[0., 2., 0.],
        [3., 0., 6.]])
```

Một block CNN cơ bản thường có dạng:

$$X \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow Y$$

Viết dưới dạng công thức:

$$Z = \text{Conv}(X; W, b)$$

sau đó

$$A = \text{ReLU}(Z)$$

Trong đó:

- Z output trước activation;
- A output sau activation.

VII

PHẦN VII

Mục 36–41 · 6 mục

Pooling

Max Pooling và Average Pooling: tính tay trên ma trận 4×4, công thức, PyTorch; pooling chỉ thu nhỏ H, W và giữ nguyên số channel.

36 Pooling

37 Max Pooling

38 Công thức Max Pooling

39 Average Pooling

40 PyTorch Max Pooling

41 Pooling có làm thay đổi số channel không?

Mục 36 Pooling

Phần VII

Pooling làm giảm kích thước không gian của feature map.

Ví dụ $32 \times 32 \to 16 \times 16$

Hai loại thường gặp:

1 **Max Pooling.** Mục 37 →

2 **Average Pooling.** Mục 39 →

Mục 37 Max Pooling

Phần VII

Max Pooling lấy giá trị lớn nhất trong mỗi vùng.

Ví dụ:

$$X = \begin{bmatrix} 1 & \boxed{7} & 2 & 4 \\ 5 & 3 & \boxed{8} & 6 \\ \boxed{9} & 2 & 4 & 7 \\ 3 & 6 & 5 & \boxed{8} \end{bmatrix}$$

Max Pool 2×2, stride 2.

Chia thành bốn vùng.

Vùng 1

$$\begin{bmatrix} 1 & \boxed{7} \\ 5 & 3 \end{bmatrix}$$

Max 7

Vùng 2

$$\begin{bmatrix} 2 & 4 \\ \boxed{8} & 6 \end{bmatrix}$$

Max 8

Vùng 3

$$\begin{bmatrix} \boxed{9} & 2 \\ 3 & 6 \end{bmatrix}$$

Max 9

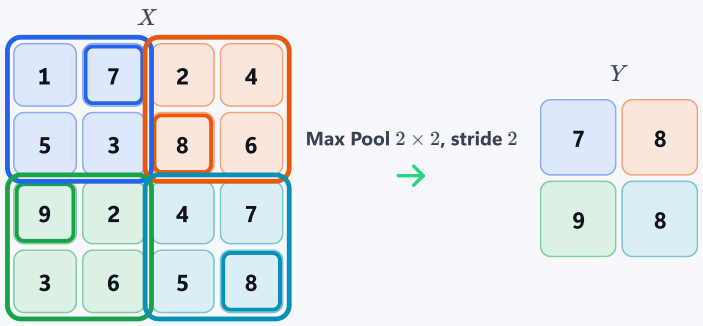
Vùng 4

$$\begin{bmatrix} 4 & 7 \\ 5 & \boxed{8} \end{bmatrix}$$

Max 8

Output:

$$Y = \begin{bmatrix} 7 & 8 \\ 9 & 8 \end{bmatrix}$$



HÌNH 37.1 Mỗi vùng 2×2 (một màu) chỉ giữ lại giá trị lớn nhất (ô có vòng đậm); ba giá trị còn lại bị bỏ. Input 4×4 thành output 2×2.

Mục 38 Công thức Max Pooling

Phần VII

Nếu cửa sổ pooling là $K_H \times K_W$:

$$Y_{i,j} = \max_{u,v} X_{iS+u,jS+v}$$

Trong đó $0 \le u < K_H \quad 0 \le v < K_W$

Mục 39 Average Pooling

Phần VII

Average Pooling lấy trung bình.

1. **Ví dụ** $\begin{bmatrix} 1 & 3 \\ 5 & 7 \end{bmatrix}$
2. **Ta có** $\dfrac{1+3+5+7}{4} = 4$
3. **Output** 4



HÌNH 39.1 Cùng một vùng 2×2: Max Pooling giữ giá trị lớn nhất 7, Average Pooling lấy trung bình $\frac{1+3+5+7}{4} = 4$ (bài giảng bổ sung phép so sánh).

Mục 40 PyTorch Max Pooling

Phần VII

PYTHON 19 dòng

```python
import torch
import torch.nn.functional as F

x = torch.tensor([
    [1., 7., 2., 4.],
    [5., 3., 8., 6.],
    [9., 2., 4., 7.],
    [3., 6., 5., 8.]
])

x = x.unsqueeze(0).unsqueeze(0)

y = F.max_pool2d(
    x,
    kernel_size=2,
    stride=2
)

print(y)
```

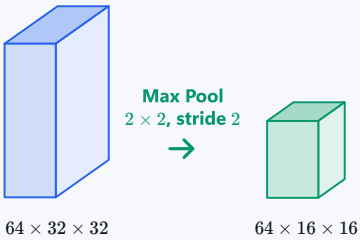
KẾT QUẢ

```
tensor([[[[7., 8.],
          [9., 8.]]]])
```

Mục 41 Pooling có làm thay đổi số channel không?

Phần VII

1	Thông thường	$C_{\text{out}} = C_{\text{in}}$
2	Pooling chỉ thay đổi	H, W
3	Ví dụ	$64 \times 32 \times 32$
4	Max Pool 2×2, stride 2	$64 \times 16 \times 16$
5	Số channel vẫn là	64



HÌNH 41.1 Chiều rộng của khối là số channel, mặt trước là $H \times W$. Pooling làm mỗi feature map nhỏ đi một nửa theo mỗi chiều, còn số feature map (64) giữ nguyên vì mỗi channel được pooling riêng.

VIII

PHẦN VIII

Mục 42–45 · 4 mục

Xếp chồng nhiều tầng

Block Conv → ReLU → Pooling điển hình; shape qua nhiều tầng; vì sao spatial size giảm dần còn số channel tăng dần; receptive field.

42 Một CNN block điển hình

43 CNN nhiều tầng

44 Tại sao channel thường tăng?

45 Receptive Field trực quan

Mục 42 Một CNN block điển hình

Phần VIII

Một block:

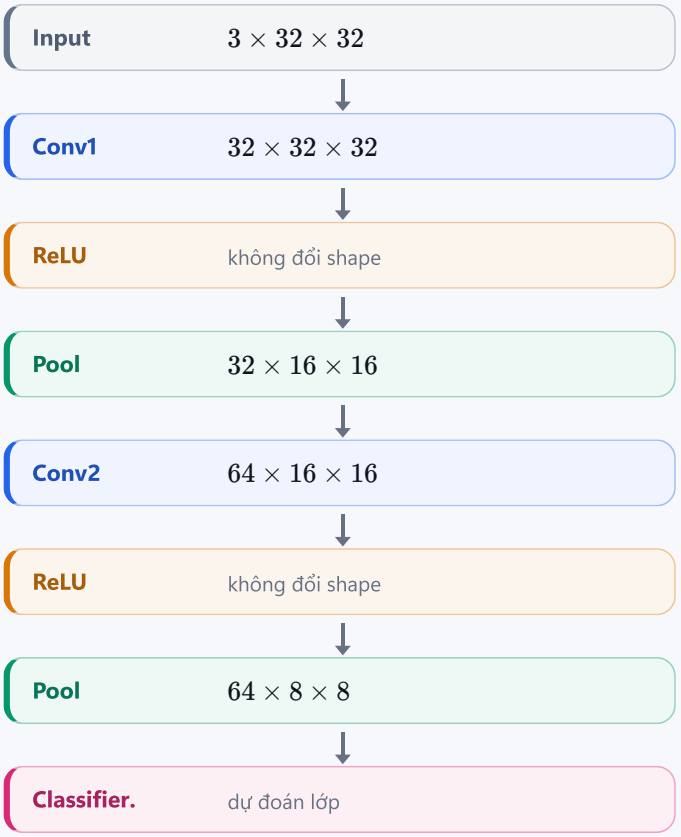
Conv → ReLU → Pooling

1. **Ví dụ** $3 \times 32 \times 32$
2. **Conv** $3 \to 32$ output $32 \times 32 \times 32$
3. **ReLU** $32 \times 32 \times 32$
4. **Max Pool** $32 \times 16 \times 16$

Mục 43 CNN nhiều tầng

Phần VIII

Ví dụ:



Mục 44 Tại sao channel thường tăng?

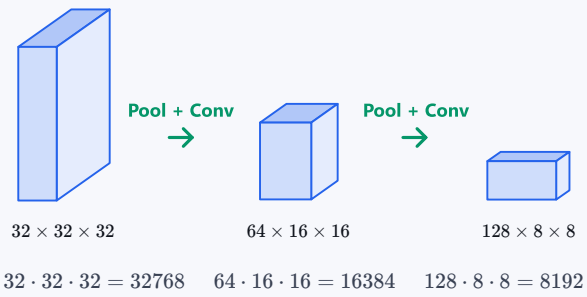
Phần VIII

Khi spatial resolution giảm:

$$32 \times 32 \to 16 \times 16 \to 8 \times 8$$

ta thường tăng số channel:

$$32 \to 64 \to 128$$



HÌNH 44.1 Theo các số của Mục 44: mặt $H \times W$ nhỏ dần ($32 \to 16 \to 8$) trong khi khối dày dần ($32 \to 64 \to 128$ channel). Hàng số bên dưới (bài giảng bổ sung) là tổng số giá trị của mỗi tensor: giảm một nửa mỗi lần.

Ý tưởng:

- spatial dimension nhỏ dần;
- số loại feature được học tăng dần.

Layer đầu có thể học các đặc trưng đơn giản.

Layer sâu hơn học những đặc trưng phức tạp hơn.

Mục 45 Receptive Field trực quan

Phần VIII

Một neuron ở layer đầu chỉ quan sát một vùng nhỏ của ảnh.

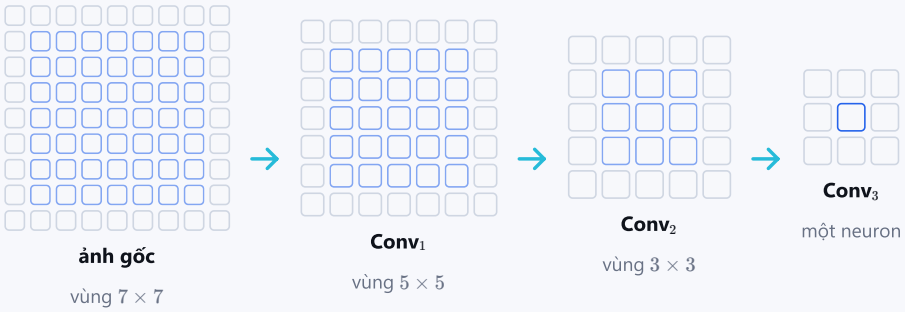
Nhưng khi xếp nhiều convolution layer:

$$\text{Conv}_1 \rightarrow \text{Conv}_2 \rightarrow \text{Conv}_3$$

một neuron ở layer sâu gián tiếp phụ thuộc vào vùng lớn hơn của ảnh gốc.

Vùng input ảnh hưởng tới một neuron được gọi là:

receptive field



HÌNH 45.1 Minh họa với ba lớp kernel 3×3, stride 1, không padding (đọc từ phải sang trái): một neuron của Conv_3 dùng 3×3 neuron của Conv_2; chúng dùng 5×5 neuron của Conv_1, và cuối cùng 7×7 pixel ảnh gốc. Receptive field tăng thêm 2 mỗi lớp.

Đây là lý do mạng sâu có thể học từ edge đến parts và cuối cùng whole objects

IX

PHẦN IX

Mục 46–54 · 9 mục

Từ feature map tới xác suất và loss

Flatten $64 \times 8 \times 8 \to 4096$, Fully Connected Layer $z = Wx + b$, logits, Softmax và Cross-Entropy tính tay; vì sao không gọi Softmax trước CrossEntropyLoss.

46 Flatten

47 PyTorch Flatten

48 Fully Connected Layer

49 Logits

50 Softmax

51 Ví dụ Softmax

52 Cross-Entropy Loss

53 Ví dụ Cross-Entropy

54 Không cần gọi Softmax trước CrossEntropyLoss trong PyTorch

Mục 46 Flatten

Phần IX

1. **Giả sử sau convolution ta có tensor** $64 \times 8 \times 8$

2. **Để đưa vào Dense Layer, có thể Flatten** $64 \times 8 \times 8$ thành vector $64 \cdot 8 \cdot 8 = 4096$

3. **Do đó** $64 \times 8 \times 8 \to 4096$



HÌNH 46.1 Minh họa với tensor nhỏ $2 \times 2 \times 2$: flatten đọc hết channel 0 (từng hàng một) rồi mới sang channel 1, được $2 \cdot 2 \cdot 2 = 8$ số. Tensor $64 \times 8 \times 8$ theo cùng quy tắc cho 4096 số.

Mục 47 PyTorch Flatten

Phần IX

PYTHON 9 dòng

```python
import torch

x = torch.randn(32, 64, 8, 8)

print(x.shape)

y = torch.flatten(x, start_dim=1)

print(y.shape)
```

OUTPUT

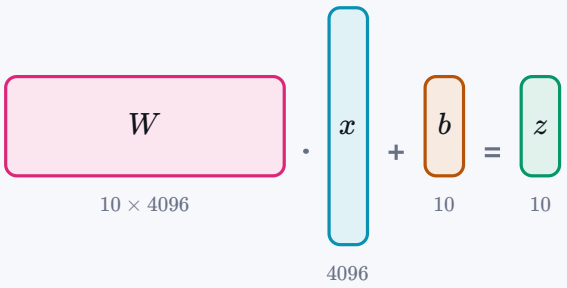
```
torch.Size([32, 64, 8, 8])
torch.Size([32, 4096])
```

Số 32 là batch size.

Mục 48 Fully Connected Layer

Phần IX

1	Sau Flatten	$x \in \mathbb{R}^{4096}$
2	Một Linear Layer	$z = Wx + b$
3	Ví dụ 10 class	$W \in \mathbb{R}^{10\times4096}$ $b \in \mathbb{R}^{10}$
4	Output	$z \in \mathbb{R}^{10}$



HÌNH 48.1 Số cột của W phải bằng số phần tử của x (4096); số hàng của W là số class (10). Mỗi hàng của W cho một logit: $z_i = \sum_k W_{ik}x_k + b_i$.

Mục 49 Logits

Phần IX

Output cuối cùng trước Softmax thường được gọi là:

$$\text{logits}$$

Ví dụ:

$$z = \begin{bmatrix} 1.2 \\ 3.1 \\ -0.7 \end{bmatrix}$$

Ba giá trị tương ứng ba class.

Logit lớn hơn thể hiện mô hình đang gán điểm số lớn hơn cho class đó.

⚠ **Logits chưa phải xác suất.**

Mục 50 Softmax

Phần IX

Softmax chuyển logits thành phân phối xác suất:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

Trong đó:

- z_i logit class i;
- C số class;
- p_i xác suất dự đoán.

Ta có $\sum_i p_i = 1$

Mục 51 Ví dụ Softmax

Phần IX

1. **Cho** $z = [1, 2, 3]$
2. **Ta tính** $e^1 \approx 2.718$ $e^2 \approx 7.389$ $e^3 \approx 20.086$
3. **Tổng** 30.193

4 **Do đó**

$$p_1 = \frac{2.718}{30.193} \approx 0.090$$
$$p_2 = \frac{7.389}{30.193} \approx 0.245$$
$$p_3 = \frac{20.086}{30.193} \approx 0.665$$

5 **Vậy** $p \approx [0.090, 0.245, 0.665]$



HÌNH 51.1 Ba bước của Mục 51: lấy e^{z_i} làm mọi số dương và khuếch đại chênh lệch (3 chỉ gấp 3 lần 1 nhưng e^3 gấp 7.4 lần e^1); chia cho tổng 30.193 để ba số cộng lại bằng 1.

Mục 52 Cross-Entropy Loss

Phần IX

Giả sử class đúng là y.

Cross-Entropy cho một mẫu:

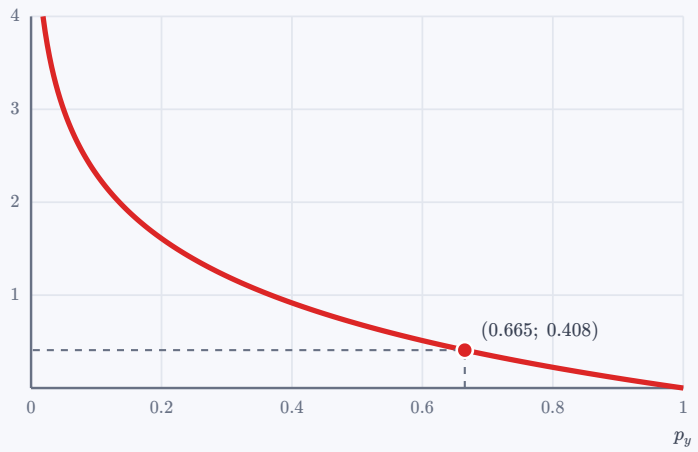
$$L = -\log p_y$$

Nếu mô hình gán xác suất cao cho class đúng

$p_y \to 1$ thì $L \to 0$

Nếu mô hình gán xác suất thấp

$p_y \to 0$ thì loss tăng mạnh.



HÌNH 52.1 $L = -\ln p_y$ theo p_y (log ở Mục 52 là logarit tự nhiên ln, như Mục 53 và `nn.CrossEntropyLoss`). Chấm đỏ là ví dụ của Mục 53: $p_3 = 0.665$ cho $L \approx 0.408$. Khi $p_y \to 0$, loss tăng không giới hạn.

Mục 53 Ví dụ Cross-Entropy

Phần IX

Nếu class đúng là class 3

$$p_3 = 0.665$$

thì

$$L = -\ln(0.665)$$

$$\approx 0.408$$

Mục 54 Không cần gọi Softmax trước CrossEntropyLoss trong PyTorch

Phần IX

Trong PyTorch

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, labels)
```

Không cần (trước đó)

```
softmax(logits)
```

`CrossEntropyLoss` nhận trực tiếp logits

X

PHẦN X

Mục 55–60 · 6 mục

CNN hoàn chỉnh

Kiến trúc đầy đủ và shape sau từng layer, lớp SimpleCNN bằng PyTorch, batch dimension $[N, C, H, W]$, cú pháp nn.Conv2d và shape của weight.

55 Kiến trúc CNN hoàn chỉnh

56 Theo dõi tensor bằng một ví dụ hoàn chỉnh

57 CNN hoàn chỉnh bằng PyTorch

58 Ý nghĩa batch dimension

59 `nn.Conv2d`

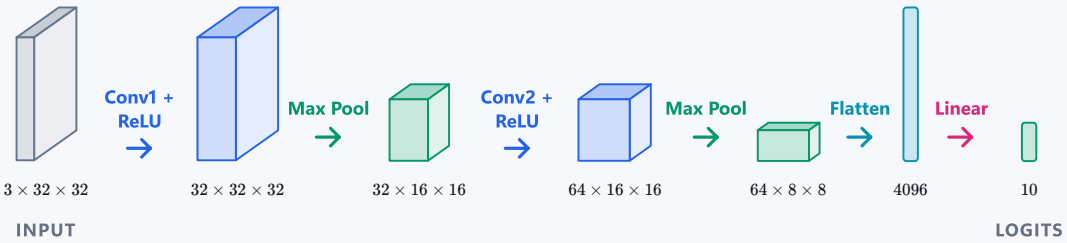
60 Kiểm tra weight

Mục 55 Kiến trúc CNN hoàn chỉnh

Phần X

Ta có:

$$X \to \text{Conv}_1 \to \text{ReLU} \to \text{Pool} \to \text{Conv}_2 \to \text{ReLU} \to \text{Pool} \to \text{Flatten} \to \text{Linear} \to \text{Logits} \to \text{Loss}$$



HÌNH 55.1 Kiến trúc của Mục 55 với shape theo dõi ở Mục 56 (ReLU không đổi shape nên vẽ chung với Conv). Độ dày của khối là số channel, mặt trước là $H \times W$; sau Flatten chỉ còn một vector. Loss được tính từ 10 logits và nhãn đúng.

Mục 56 Theo dõi tensor bằng một ví dụ hoàn chỉnh

Phần X

Input $X \in \mathbb{R}^{3\times32\times32}$

1 **Conv1**

in_channels	= 3	C_{in}
out_channels	= 32	C_{out}
kernel_size	= 3	K
stride	= 1	S
padding	= 1	P

Spatial size $H_{\text{out}} = \frac{32+2-3}{1} + 1 = 32$

Output $32 \times 32 \times 32$

2 **ReLU** — Không đổi shape $32 \times 32 \times 32$

3 **Max Pool**

Kernel 2×2

Stride 2

Output $32 \times 16 \times 16$

4 **Conv2**

in_channels	= 32	C_{in}
out_channels	= 64	C_{out}
kernel_size	= 3	K
stride	= 1	S
padding	= 1	P

Output $64 \times 16 \times 16$

5 **ReLU** — $64 \times 16 \times 16$

6 **Max Pool** — $64 \times 8 \times 8$

7 **Flatten** — $64 \times 8 \times 8 = 4096$

8 **Linear**

Nếu có 10 class $4096 \to 10$

Output 10 logits.

Mục 57 CNN hoàn chỉnh bằng PyTorch

Phần X

PYTHON 71 dòng

```python
import torch
import torch.nn as nn


class SimpleCNN(nn.Module):

    def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(
```

```
            in_channels=3,
            out_channels=32,
            kernel_size=3,
            stride=1,
            padding=1
        )

        self.relu1 = nn.ReLU()

        self.pool1 = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        self.conv2 = nn.Conv2d(
            in_channels=32,
            out_channels=64,
            kernel_size=3,
            stride=1,
            padding=1
        )

        self.relu2 = nn.ReLU()

        self.pool2 = nn.MaxPool2d(
            kernel_size=2,
            stride=2
        )

        self.flatten = nn.Flatten()

        self.fc = nn.Linear(
            64 * 8 * 8,
            num_classes
        )

    def forward(self, x):

        x = self.conv1(x)
        x = self.relu1(x)
        x = self.pool1(x)

        x = self.conv2(x)
        x = self.relu2(x)
        x = self.pool2(x)

        x = self.flatten(x)

        x = self.fc(x)

        return x


model = SimpleCNN(num_classes=10)

x = torch.randn(4, 3, 32, 32)

```

```python
logits = model(x)

print("Input :", x.shape)
print("Output:", logits.shape)
```

KẾT QUẢ

```
Input : torch.Size([4, 3, 32, 32])
Output: torch.Size([4, 10])
```

Mục 58 Ý nghĩa batch dimension

Phần X

Trong PyTorch, image tensor thường có dạng:

$$[N, C, H, W]$$

Trong đó:

- N batch size;
- C channel;
- H height;
- W width.

Ví dụ:

$$[\ 32\ ,\ 3\ ,\ 224\ ,\ 224\ ]$$

N C H W

nghĩa là:

32 ảnh mỗi ảnh 3 channel cao 224 rộng 224

Mục 59 nn.Conv2d

Phần X

Cú pháp thường gặp:

PYTHON 7 dòng

```python
nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size,
    stride=1,
    padding=0
)
```

Ví dụ:

PYTHON 7 dòng

```python
nn.Conv2d(
    in_channels=3,
    out_channels=64,
    kernel_size=3,
    stride=1,
    padding=1
)
```

Ý nghĩa:

`in_channels=3`	→	3 input channel.
`out_channels=64`	→	64 filter.
`kernel_size=3`	→	Mỗi filter có kích thước $3 \times 3 \times 3$
`out_channels=64`	→	Output có 64 channel.

Mục 60 Kiểm tra weight

Phần X

PYTHON 11 dòng

```python
import torch.nn as nn

conv = nn.Conv2d(
    3,
    64,
    kernel_size=3,
    padding=1
)

print(conv.weight.shape)
print(conv.bias.shape)
```

OUTPUT

```
torch.Size([64, 3, 3, 3])
torch.Size([64])
```

Do đó $W \in \mathbb{R}^{64\times3\times3\times3}$ và $b \in \mathbb{R}^{64}$

XI

PHẦN XI

Mục 61–69 · 9 mục

CNN học như thế nào

Forward, loss, backpropagation, cập nhật weight và bias; optimizer SGD, Adam; một bước training năm dòng; chương trình huấn luyện hoàn chỉnh trên CIFAR-10 và argmax.

61 CNN học filter như thế nào?

62 Forward Pass

63 Backpropagation

64 Cập nhật bias

65 Kernel không được thiết kế thủ công trong CNN hiện đại

66 Optimizer

67 Một bước training hoàn chỉnh

68 Chương trình training CNN hoàn chỉnh trên CIFAR-10

69 Giải thích `argmax`

Mục 61 CNN học filter như thế nào?

Phần XI

Ban đầu các weight thường được khởi tạo bằng các giá trị phù hợp nhưng chưa chứa tri thức về bài toán.

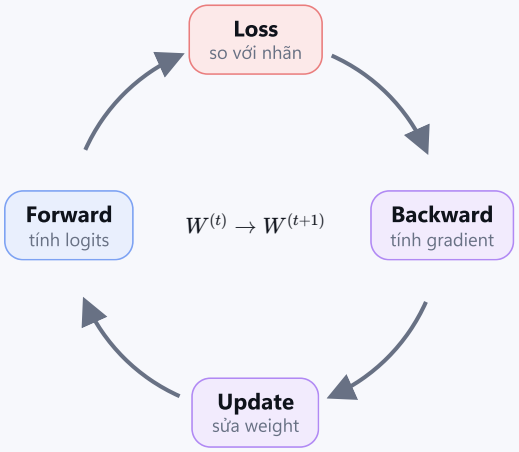
Trong quá trình training:

$$W^{(0)} \rightarrow W^{(1)} \rightarrow W^{(2)} \rightarrow \dots$$

Các weight được điều chỉnh để giảm loss.

Quy trình:

Forward → Loss → Backward → Update



HÌNH 61.1 Mỗi vòng dùng một batch ảnh: Forward tính logits, Loss so với nhãn, Backward tính gradient, Update sửa weight từ $W^{(t)}$ thành $W^{(t+1)}$. Lặp lại vòng này rất nhiều lần cho ra chuỗi $W^{(0)} \to W^{(1)} \to W^{(2)} \to \cdots$ của Mục 61.

Mục 62 Forward Pass

Phần XI

1	**Input**	X	được truyền qua toàn bộ network	$X \to \text{Conv} \to \text{ReLU} \to \text{Pool} \to \cdots \to z$
2	**Ta nhận logits**	z		
3	**Sau đó tính**	$L(z, y)$		

Mục 63 Backpropagation

Phần XI

Ta cần biết loss thay đổi thế nào khi weight thay đổi.

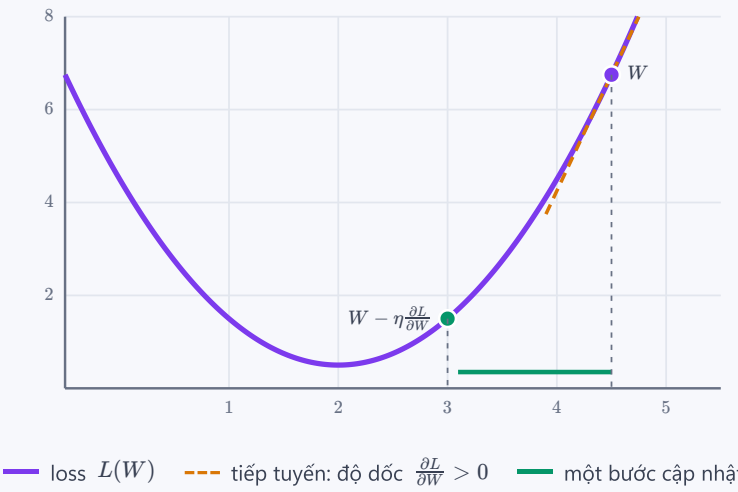
Ví dụ $\frac{\partial L}{\partial W}$

Đây là gradient của loss theo weight.

Optimizer sử dụng gradient để cập nhật:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Trong đó η là learning rate.



HÌNH 63.1 Minh họa với $L(W) = (W-2)^2 + 0.5$, $\eta = 0.3$: tại $W = 4.5$ độ dốc $\frac{\partial L}{\partial W} = 5 > 0$, nên bước cập nhật trừ đi $0.3 \times 5 = 1.5$ và đưa W về 3, loss giảm từ 6.75 còn 1.5. Weight luôn đi ngược dấu gradient, về phía loss nhỏ hơn.

Mục 64 Cập nhật bias

Phần XI

Tương tự:

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Do đó cả:

- **convolution weight**
- **convolution bias**
- **fully connected weight**
- **fully connected bias**

đều có thể được học.

Mục 65 Kernel không được thiết kế thủ công trong CNN hiện đại

Phần XI

Điểm quan trọng:

CNN KHÔNG HOẠT ĐỘNG THEO CÁCH

Con người viết sẵn kernel phát hiện mắt.

Thay vào đó W là trainable parameter.

☆ **Mạng tự tìm những filter hữu ích để giảm loss.**

Mục 66 Optimizer

Phần XI

Ví dụ SGD

```
optimizer = torch.optim.SGD(
    model.parameters(),
    lr=0.01
)
```

Hoặc Adam

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

Mục 67 Một bước training hoàn chỉnh

Phần XI

PYTHON 9 dòng

```python
optimizer.zero_grad()

logits = model(images)

loss = loss_fn(logits, labels)

loss.backward()

optimizer.step()
```

Ý nghĩa:

1. `optimizer.zero_grad()` xóa gradient cũ.
2. `logits = model(images)` forward pass. Mục 62
3. `loss = loss_fn(logits, labels)` tính loss. Mục 52
4. `loss.backward()` backpropagation. Mục 63
5. `optimizer.step()` cập nhật parameters. Mục 66

Mục 68 Chương trình training CNN hoàn chỉnh trên CIFAR-10

Phần XI

PYTHON 145 dòng

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision import transforms


class SimpleCNN(nn.Module):

    def __init__(self):
        super().__init__()
```

```
12
13          self.features = nn.Sequential(
14
15              nn.Conv2d(
16                  in_channels=3,
17                  out_channels=32,
18                  kernel_size=3,
19                  padding=1
20              ),
21
22              nn.ReLU(),
23
24              nn.MaxPool2d(
25                  kernel_size=2,
26                  stride=2
27              ),
28
29              nn.Conv2d(
30                  in_channels=32,
31                  out_channels=64,
32                  kernel_size=3,
33                  padding=1
34              ),
35
36              nn.ReLU(),
37
38              nn.MaxPool2d(
39                  kernel_size=2,
40                  stride=2
41              )
42          )
43
44          self.classifier = nn.Sequential(
45
46              nn.Flatten(),
47
48              nn.Linear(
49                  64 * 8 * 8,
50                  128
51              ),
52
53              nn.ReLU(),
54
55              nn.Linear(
56                  128,
57                  10
58              )
59          )
60
61      def forward(self, x):
62
63          x = self.features(x)
64
65          x = self.classifier(x)
66
67          return x
68
```

```
69
70  transform = transforms.ToTensor()
71
72  train_dataset = datasets.CIFAR10(
73      root="./data",
74      train=True,
75      download=True,
76      transform=transform
77  )
78
79  train_loader = DataLoader(
80      train_dataset,
81      batch_size=64,
82      shuffle=True
83  )
84
85  device = torch.device(
86      "cuda" if torch.cuda.is_available() else "cpu"
87  )
88
89  model = SimpleCNN().to(device)
90
91  loss_fn = nn.CrossEntropyLoss()
92
93  optimizer = torch.optim.Adam(
94      model.parameters(),
95      lr=0.001
96  )
97
98
99  for epoch in range(5):
100
101     model.train()
102
103     total_loss = 0.0
104     total_correct = 0
105     total_samples = 0
106
107     for images, labels in train_loader:
108
109         images = images.to(device)
110         labels = labels.to(device)
111
112         optimizer.zero_grad()
113
114         logits = model(images)
115
116         loss = loss_fn(
117             logits,
118             labels
119         )
120
121         loss.backward()
122
123         optimizer.step()
124
125         total_loss += loss.item() * images.size(0)
```

```
126
127         predictions = logits.argmax(dim=1)
128
129         total_correct += (
130             predictions == labels
131         ).sum().item()
132
133         total_samples += labels.size(0)
134
135     average_loss = total_loss / total_samples
136
137     accuracy = (
138         total_correct / total_samples
139     )
140
141     print(
142         f"Epoch {epoch + 1}: "
143         f"loss={average_loss:.4f}, "
144         f"accuracy={accuracy:.4f}"
145     )
```

KẾT QUẢ MỘT LẦN CHẠY THỬ (BÀI GIẢNG BỔ SUNG)

```
Epoch 1: loss=1.4801, accuracy=0.4659
Epoch 2: loss=1.1203, accuracy=0.6041
Epoch 3: loss=0.9727, accuracy=0.6562
Epoch 4: loss=0.8765, accuracy=0.6910
Epoch 5: loss=0.8009, accuracy=0.7194
```

GHI CHÚ CỦA BÀI GIẢNG

Chương trình trên đã được chạy nguyên văn với PyTorch 2.13.0 chỉ trên CPU (AMD Ryzen 9 8945HS, 8 nhân). Lần chạy đầu tải CIFAR-10 (khoảng 170 MB) vào thư mục `./data`; sau đó mỗi epoch mất chừng 19 giây. Trên GPU nhanh hơn nhiều.

Weight được khởi tạo ngẫu nhiên và dữ liệu được xáo trộn (`shuffle=True`) nên mỗi lần chạy cho số liệu hơi khác: lần chạy thứ hai kết thúc với `loss=0.7903, accuracy=0.7227`. Xu hướng thì giống nhau: loss giảm, accuracy tăng qua các epoch. Accuracy này đo trên chính tập train; Mục 77 và Mục 78 giải thích vì sao cần thêm tập validation.

Mục 69 Giải thích `argmax`

Phần XI

Giả sử logits $[1.4, 0.3, 4.8, 2.1]$

Phần tử lớn nhất là 4.8 tại vị trí 2

Do đó:

PYTHON

```
1 logits.argmax(dim=1)
```

→ trả về class 2



HÌNH 69.1 Chỉ số đếm từ 0, nên 4.8 nằm ở vị trí 2 (class thứ ba). Trong chương trình, `logits` có shape $[N, 10]$: `argmax(dim=1)` lấy chỉ số lớn nhất trên từng hàng, tức một dự đoán cho mỗi ảnh.

XII

PHẦN XII

Mục 70–75 · 6 mục

Thành phần của CNN hiện đại

Global Average Pooling thay cho Flatten lớn, Batch Normalization, Dropout và Data Augmentation.

70 Global Average Pooling

71 Ví dụ Global Average Pooling

72 PyTorch Global Average Pooling

73 Batch Normalization

74 Dropout

75 Data Augmentation

Mục 70 Global Average Pooling

Phần XII

CNN hiện đại thường không nhất thiết Flatten toàn bộ feature map.

1 **Giả sử** $X \in \mathbb{R}^{C\times H\times W}$

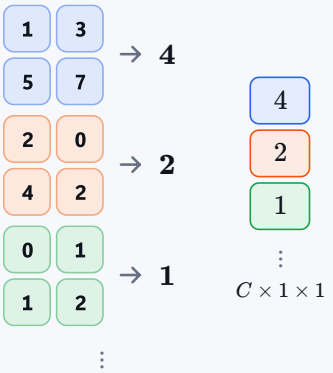
2 **Global Average Pooling lấy trung bình mỗi channel**

$$y_c = \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W} X_{c,i,j}$$

3 **Output** $C \times 1 \times 1$

hoặc sau khi bỏ hai chiều cuối

C



HÌNH 70.1 Mỗi channel (một feature map $H \times W$) được thay bằng trung bình của nó; feature map đầu là ví dụ của Mục 71 (4), hai feature map sau là số minh họa. C channel cho C số, bất kể H và W lớn bao nhiêu.

Mục 71 Ví dụ Global Average Pooling

Phần XII

Feature map $X = \begin{bmatrix} 1 & 3 \\ 5 & 7 \end{bmatrix}$

Trung bình $\frac{1+3+5+7}{4} = 4$

Một channel được biến thành một số.

Nếu có 128 channel thì output vector có 128 phần tử.

Mục 72 PyTorch Global Average Pooling

Phần XII

Có thể sử dụng:

PYTHON

```python
nn.AdaptiveAvgPool2d((1, 1))
```

Ví dụ:

PYTHON 10 dòng

```python
import torch
import torch.nn as nn

x = torch.randn(32, 128, 8, 8)

gap = nn.AdaptiveAvgPool2d((1, 1))

y = gap(x)

print(y.shape)
```

OUTPUT

```
torch.Size([32, 128, 1, 1])
```

Sau Flatten:

PYTHON

```python
y = torch.flatten(y, 1)
```

SHAPE

```
torch.Size([32, 128])
```

Mục 73 Batch Normalization

Phần XII

Một thành phần thường gặp trong CNN hiện đại là Batch Normalization.

Block có thể có dạng:



Trong PyTorch:

```python
nn.Conv2d(32, 64, 3, padding=1)
nn.BatchNorm2d(64)
nn.ReLU()
```

BatchNorm hỗ trợ ổn định quá trình optimization trong nhiều kiến trúc.

Mục 74 Dropout

Phần XII

Dropout là một regularization technique.

Trong training, một số activation được đặt thành 0 ngẫu nhiên.

Ví dụ:

```python
nn.Dropout(0.5)
```

có thể được sử dụng trong classifier.

Dropout chỉ hoạt động theo cơ chế dropout khi

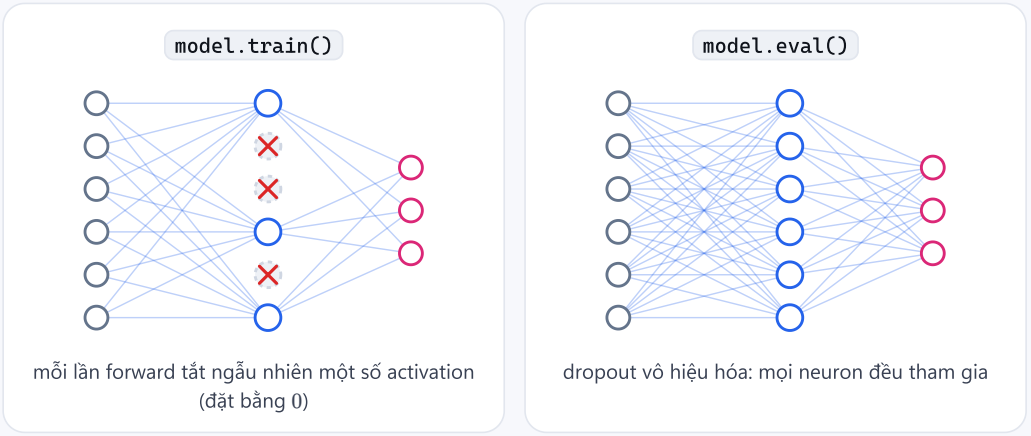
```python
model.train()
```

dropout hoạt động

Trong evaluation

```python
model.eval()
```

Dropout bị vô hiệu hóa



mỗi lần forward tắt ngẫu nhiên một số activation (đặt bằng 0)

dropout vô hiệu hóa: mọi neuron đều tham gia

HÌNH 74.1 Minh họa với `nn.Dropout(0.5)` trên một lớp ẩn: mỗi lần forward khi training, mỗi activation bị đặt thành 0 với xác suất 0.5 (lần này ba neuron có dấu ×), nên mạng không thể dựa vào một neuron riêng lẻ. Khi `model.eval()`, không neuron nào bị tắt.

Mục 75 Data Augmentation

Phần XII

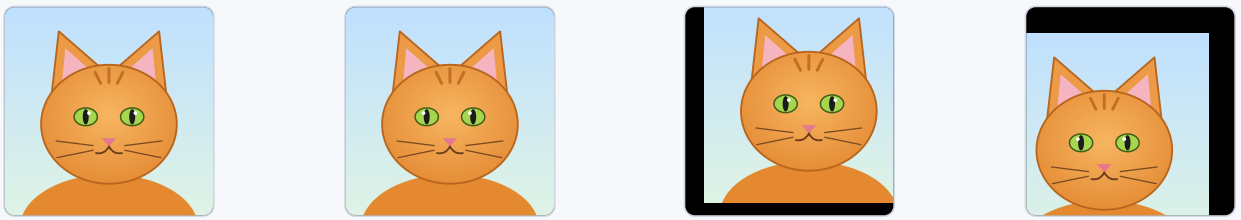
CNN có thể được cải thiện bằng cách tạo biến thể của training images.

Ví dụ:

PYTHON 8 dòng

```python
transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(
        32,
        padding=4
    ),
    transforms.ToTensor()
])
```

Mục đích là làm cho training distribution đa dạng hơn và giúp mô hình tổng quát hóa tốt hơn.



ảnh gốc

`RandomHorizontalFlip()` : lật ngang

`RandomCrop(32, padding=4)` : dịch và có viền đen

cả hai

HÌNH 75.1 Minh họa trên hình vẽ con mèo; ảnh CIFAR-10 có kích thước 32×32. `RandomCrop(32, padding=4)` thêm viền 4 pixel (giá trị 0, màu đen) rồi cắt một ô 32×32 ở vị trí ngẫu nhiên, nên ảnh bị dịch tối đa 4 pixel. Mỗi epoch, cùng một ảnh train được biến đổi ngẫu nhiên theo một cách khác.

XIII

PHẦN XIII

Mục 76–79 · 4 mục

Huấn luyện và đánh giá

Train mode và eval mode, tập validation, nhận biết overfitting và underfitting cùng các biện pháp thường gặp.

76 Train mode và eval mode

77 Validation

78 Overfitting

79 Underfitting

Mục 76 Train mode và eval mode

Phần XIII

Khi training

PYTHON

```python
model.train()
```

Khi validation hoặc testing

PYTHON

```python
model.eval()
```

Trong evaluation thường sử dụng:

PYTHON

```python
with torch.no_grad():
```

Ví dụ:

PYTHON 7 dòng

```python
model.eval()

with torch.no_grad():

    logits = model(images)

    predictions = logits.argmax(dim=1)
```

Mục 77 Validation

Phần XIII

Không nên chỉ quan sát training accuracy.

Cần chia dữ liệu thành train và validation

Training set

Training set dùng để cập nhật weight.

Validation set

Validation set dùng để đánh giá khả năng generalization trong quá trình phát triển mô hình.

Mục 78 Overfitting

Phần XIII

Nếu training accuracy rất cao nhưng validation accuracy thấp mô hình có thể đang overfit.

Các biện pháp thường gặp:

- data augmentation;
- weight decay;
- dropout;
- nhiều dữ liệu hơn;
- điều chỉnh độ phức tạp mô hình;
- early stopping.

Mục 79 Underfitting

Phần XIII

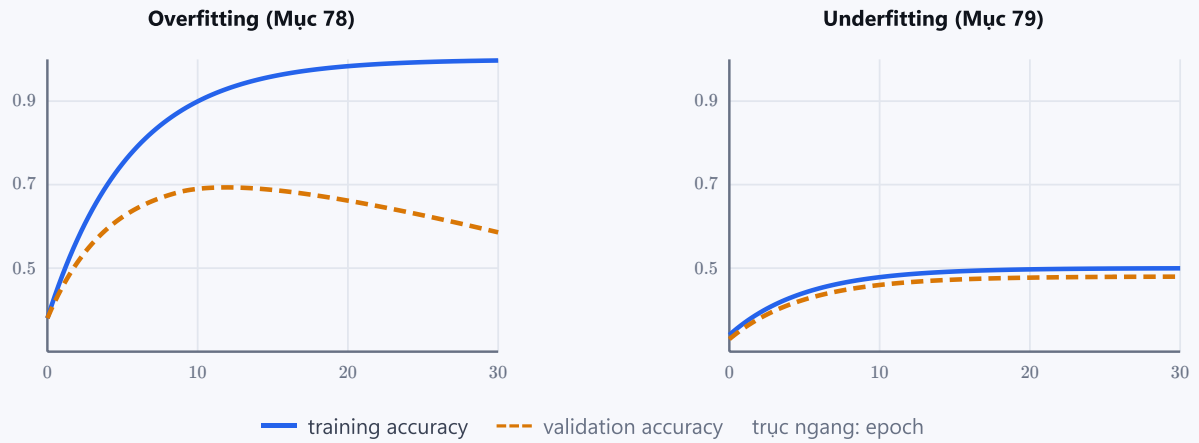
Nếu cả:

- training accuracy thấp;
- validation accuracy thấp,

thì mô hình có thể chưa học đủ.

Nguyên nhân có thể gồm:

- mô hình quá nhỏ;
- training chưa đủ;
- learning rate chưa phù hợp;
- pipeline dữ liệu có vấn đề;
- optimization chưa tốt.



HÌNH 79.1 Đường minh họa (không phải số liệu đo). Overfitting: training accuracy tiến gần 1 trong khi validation accuracy dừng lại rồi giảm, khoảng cách hai đường ngày càng lớn. Underfitting: cả hai đường đều thấp và gần nhau.

XIV

PHẦN XIV
Mục 80–86 · 7 mục

Hiểu sâu CNN

Pattern channel tăng, spatial giảm; học đặc trưng theo cấp độ; convolution là phép biến đổi tuyến tính cục bộ $y = w^T x + b$; vì sao CNN ít tham số hơn Dense Network rất nhiều.

80 CNN sâu hơn
81 CNN học đặc trưng theo cấp độ
82 Convolution là phép Linear Transformation cục bộ
83 Ví dụ khai triển một patch RGB
84 Hai ý tưởng cốt lõi làm CNN hiệu quả
85 Ví dụ so sánh số tham số
86 Một CNN tốt không nhất thiết cần Flatten lớn

Mục 80 CNN sâu hơn

Phần XIV

1. **Một pattern rất phổ biến** $3 \to 64 \to 128 \to 256 \to 512$
2. **Trong khi spatial size** $224 \to 112 \to 56 \to 28 \to 14 \to 7$
3. **Tức là** $H, W \downarrow \qquad C \uparrow$

Mục 81 CNN học đặc trưng theo cấp độ

Phần XIV

Có thể hình dung:

Layer đầu

Học

edge · gradient · color transition

Layer giữa

Học

texture · simple shapes · repeated patterns

Layer sâu

Học các cấu trúc phức tạp hơn liên quan đến bài toán.

ⓘ **Điều này không phải luật tuyệt đối cho mọi neuron, nhưng là một cách hiểu hữu ích về hierarchical representation learning.**

Mục 82 Convolution là phép Linear Transformation cục bộ

Phần XIV

Nếu lấy riêng một patch

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

và Flatten kernel thành

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

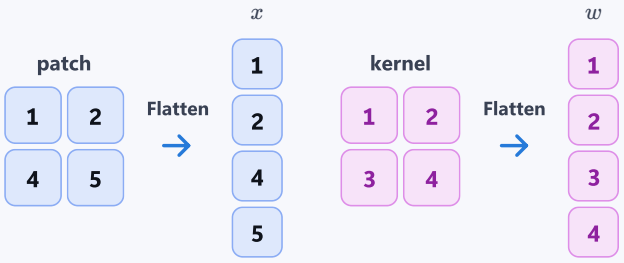
thì convolution tại một vị trí thực chất là:

$$y = w^T x + b$$

hay

$$y = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

☆ Convolution vì vậy có thể xem như cùng một Linear Layer nhỏ được áp dụng lặp lại trên nhiều vị trí.



$$w^T x = 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 4 + 4 \cdot 5 = 37$$

HÌNH 82.1 Dùng lại số của Mục 7: patch $\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$ và kernel $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ flatten cùng một thứ tự; $w^T x = 37$ đúng bằng $Y_{1,1}$ (ví dụ đó không có bias, $b = 0$).

Mục 83 Ví dụ khai triển một patch RGB

Phần XIV

1	**Kernel**	3×3
2	**Input channel**	3
3	**Một patch chứa**	$3 \times 3 \times 3 = 27$ giá trị.
4	**Ta Flatten thành**	$x \in \mathbb{R}^{27}$
5	**Filter**	$w \in \mathbb{R}^{27}$
6	**Một output pixel**	$y = w^T x + b$ hay $y = \sum_{k=1}^{27} w_k x_k + b$

☆ Đây chính là phép convolution tại một vị trí.

Mục 84 Hai ý tưởng cốt lõi làm CNN hiệu quả

Phần XIV

CNN dựa mạnh vào hai nguyên lý:

Local connectivity

Mỗi neuron chỉ nhìn một vùng nhỏ.

Mục 30

Weight sharing

Cùng một kernel được sử dụng trên nhiều vị trí.

Mục 29

☆ Nhờ đó, CNN có số tham số nhỏ hơn rất nhiều so với Dense Network xử lý trực tiếp toàn bộ ảnh.

Mục 85 Ví dụ so sánh số tham số

Phần XIV

Dense layer

Ảnh

$32 \times 32 \times 3$

Flatten

3072

Dense layer có 64 neuron

$$3072 \times 64 + 64 = 196672$$

tham số.

Conv2d

Trong khi Conv2d

$3 \to 64$

kernel

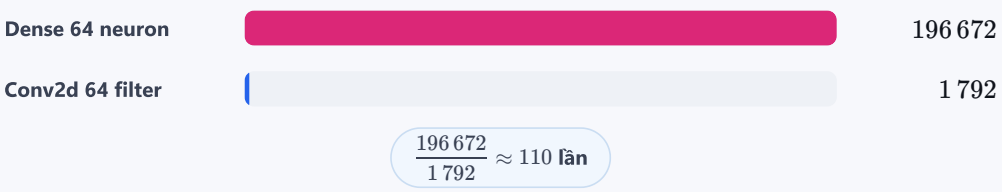
3×3

số tham số

$$64(3 \times 3 \times 3 + 1) = 64(28) = \boxed{1792}$$

Khác biệt rất lớn:

$$196672 \quad \text{so với} \quad 1792$$



HÌNH 85.1 Hai thanh vẽ đúng tỉ lệ: thanh của Conv2d chỉ dài chưa tới 1% thanh của Dense layer. Dense layer còn tăng theo kích thước ảnh, trong khi số tham số của Conv2d chỉ phụ thuộc $C_{\text{in}}, C_{\text{out}}, K_H, K_W$.

Mục 86 Một CNN tốt không nhất thiết cần Flatten lớn

Phần XIV

Nếu Flatten $512 \times 7 \times 7$ ta có 25088 feature.

Nếu nối vào Dense 4096 neuron thì số weight rất lớn.

Đó là một lý do nhiều CNN hiện đại sử dụng Global Average Pooling để giảm số parameter ở classifier.

TÍNH THỬ (BÀI GIẢNG BỔ SUNG)

Nối 25088 feature vào Dense 4096 neuron cần $25088 \times 4096 = 102\,760\,448$ weight (chưa kể 4096 bias).

Nếu dùng Global Average Pooling trước: $512 \times 7 \times 7 \to 512$, cùng Dense 4096 neuron chỉ cần $512 \times 4096 = 2\,097\,152$ weight, ít hơn 49 lần.

XV

PHẦN XV

Mục 87–92 · 6 mục

Kiến trúc nổi tiếng và Residual Connection

Các kiến trúc quan trọng từ LeNet tới EfficientNet; residual connection $Y = F(X) + X$ của ResNet và một residual block bằng PyTorch; pipeline classification tổng quát; CNN trong các bài toán khác.

87 Các kiến trúc CNN quan trọng trong lịch sử Deep Learning

88 Residual Connection

89 Một Residual Block đơn giản

90 Tại sao hai tensor cộng được?

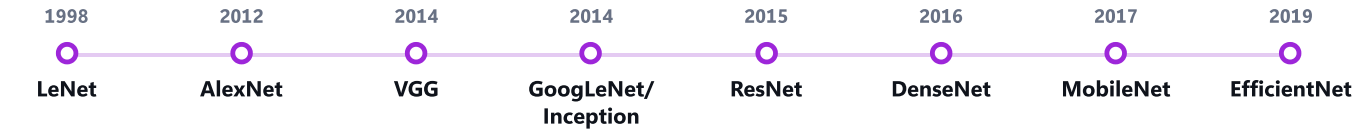
91 CNN classification pipeline đầy đủ

92 CNN không chỉ dùng cho classification

Mục 87 Các kiến trúc CNN quan trọng trong lịch sử Deep Learning

Phần XV

Người học CNN nên biết tên một số architecture quan trọng:



Năm ghi cạnh mỗi tên là năm công bố bài báo giới thiệu kiến trúc (bài giảng bổ sung).

Điều quan trọng ở giai đoạn đầu không phải học thuộc tất cả, mà hiểu chắc các building block:

1. Conv
2. Activation
3. Downsampling
4. Normalization
5. Residual connection
6. Classifier

Mục 88 Residual Connection

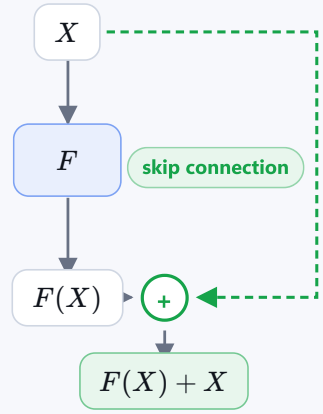
Phần XV

Một ý tưởng cực kỳ quan trọng trong CNN sâu là:

$$Y = F(X) + X$$

Thay vì block chỉ học $Y = F(X)$ ta cộng input trực tiếp vào output.

☆ **Đây là nguyên lý trung tâm của ResNet.**



Sơ đồ $X \to F(X)$

đồng thời $X \longrightarrow$ skip connection

sau đó $F(X) + X$

Residual connection giúp việc optimization các mạng rất sâu trở nên khả thi hơn.

Mục 89 Một Residual Block đơn giản

Phần XV

PYTHON 54 dòng

```python
import torch
import torch.nn as nn


class ResidualBlock(nn.Module):

    def __init__(self, channels):
        super().__init__()

        self.conv1 = nn.Conv2d(
            channels,
            channels,
            kernel_size=3,
            padding=1
        )

        self.bn1 = nn.BatchNorm2d(channels)

        self.relu = nn.ReLU()

        self.conv2 = nn.Conv2d(
            channels,
            channels,
            kernel_size=3,
            padding=1
        )

        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):

        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
```

```
        out = self.relu(out)

        out = self.conv2(out)
        out = self.bn2(out)

        out = out + identity

        out = self.relu(out)

        return out


x = torch.randn(8, 64, 32, 32)

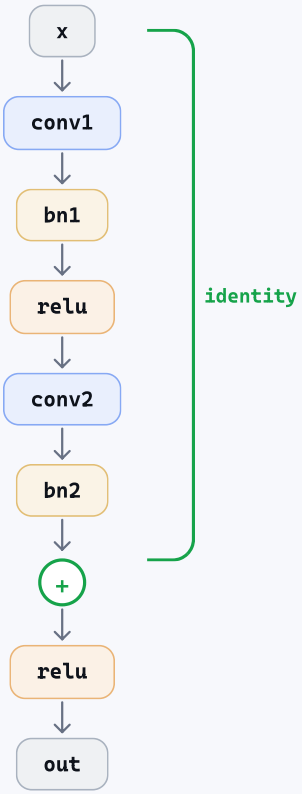
block = ResidualBlock(64)

y = block(x)

print(y.shape)
```

OUTPUT

```
torch.Size([8, 64, 32, 32])
```



HÌNH 89.1 Sơ đồ của lớp `ResidualBlock`: nhánh chính gồm hai lần Conv3×3 (padding 1) với BatchNorm, nhánh phụ là `identity = x`. Hai nhánh có cùng shape $64 \times 32 \times 32$ nên cộng được (Mục 90); ReLU cuối được áp dụng sau phép cộng.

Mục 90 Tại sao hai tensor cộng được?

Phần XV

Trong $F(X) + X$ hai tensor phải có shape tương thích.

Trong ví dụ $X : 64 \times 32 \times 32$ và $F(X) : 64 \times 32 \times 32$ nên có thể cộng từng phần tử.

Mục 91 CNN classification pipeline đầy đủ

Phần XV

Có thể tổng quát hóa:

Image → Feature Extractor → Feature Vector → Classifier → Logits

Feature Extractor có thể gồm nhiều:

Conv + Activation + Normalization + Downsampling

Classifier biến feature thành class scores.

Mục 92 CNN không chỉ dùng cho classification

Phần XV

Convolutional networks còn được sử dụng hoặc đóng vai trò backbone trong nhiều bài toán:

1. **image classification**
2. **object detection**
3. **semantic segmentation**
4. **instance segmentation**
5. **pose estimation**
6. **medical imaging**
7. **OCR**
8. **image restoration**
9. **super-resolution**
10. **image generation**

Kiến thức convolution cơ bản vẫn giữ nguyên.

XVI

PHẦN XVI

Mục 93–96 · 4 mục

Công thức, shape và lỗi thường gặp

Những công thức phải thuộc (kích thước output, số tham số, ReLU, Linear, Softmax, Cross-Entropy, gradient descent), bảng theo dõi shape, chương trình in shape tự động và sáu lỗi người mới hay mắc.

93 Những công thức phải thuộc và hiểu

94 Bảng theo dõi shape cần biết đọc

95 Chương trình theo dõi shape tự động

96 Những lỗi người mới thường gặp

Mục 93 Những công thức phải thuộc và hiểu

Phần XVI

1 Output height — Mục 16

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_H - K_H}{S_H} \right\rfloor + 1$$

2 Output width — Mục 16

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_W - K_W}{S_W} \right\rfloor + 1$$

3 Số parameter — Mục 26

$$N_{\text{params}} = C_{\text{out}}(C_{\text{in}} K_H K_W + 1)$$

4 ReLU — Mục 32

$$\text{ReLU}(x) = \max(0, x)$$

5 Linear — Mục 48

$$z = Wx + b$$

6 Softmax — Mục 50

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

7 Cross-Entropy — Mục 52

$$L = -\log p_y$$

8 Gradient descent — Mục 63

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

Mục 94 Bảng theo dõi shape cần biết đọc

Phần XVI

Giả sử batch size $N = 32$

Layer	Tensor shape
Input	$32 \times 3 \times 32 \times 32$
Conv1	$32 \times 32 \times 32 \times 32$
ReLU	$32 \times 32 \times 32 \times 32$
Pool1	$32 \times 32 \times 16 \times 16$
Conv2	$32 \times 64 \times 16 \times 16$
ReLU	$32 \times 64 \times 16 \times 16$
Pool2	$32 \times 64 \times 8 \times 8$
Flatten	32×4096
Linear	32×10

N batch C channel H height W width số feature

☆ Đây là kỹ năng cực kỳ quan trọng khi lập trình CNN.

Mục 95 Chương trình theo dõi shape tự động

Phần XVI

PYTHON 58 dòng

```python
import torch
import torch.nn as nn


conv1 = nn.Conv2d(
    3,
    32,
    kernel_size=3,
    padding=1
)

relu = nn.ReLU()

pool = nn.MaxPool2d(
    kernel_size=2,
    stride=2
)

conv2 = nn.Conv2d(
    32,
    64,
    kernel_size=3,
    padding=1
)

x = torch.randn(
    32,
```

```
    3,
    32,
    32
)

print("Input :", x.shape)

x = conv1(x)
print("Conv1 :", x.shape)

x = relu(x)
print("ReLU1 :", x.shape)

x = pool(x)
print("Pool1 :", x.shape)

x = conv2(x)
print("Conv2 :", x.shape)

x = relu(x)
print("ReLU2 :", x.shape)

x = pool(x)
print("Pool2 :", x.shape)

x = torch.flatten(
    x,
    start_dim=1
)

print("Flatten:", x.shape)
```

KẾT QUẢ KHI CHẠY (BÀI GIẢNG BỔ SUNG)

```
Input : torch.Size([32, 3, 32, 32])
Conv1 : torch.Size([32, 32, 32, 32])
ReLU1 : torch.Size([32, 32, 32, 32])
Pool1 : torch.Size([32, 32, 16, 16])
Conv2 : torch.Size([32, 64, 16, 16])
ReLU2 : torch.Size([32, 64, 16, 16])
Pool2 : torch.Size([32, 64, 8, 8])
Flatten: torch.Size([32, 4096])
```

1 Lỗi 1: nhầm số filter với kích thước kernel

```python
nn.Conv2d(3, 64, 3)
```

✗ không có nghĩa kernel 64×3.

✓ Nó có nghĩa:

- input channel = 3;
- số filter = 64;
- mỗi filter có spatial size 3×3.

Weight shape $64 \times 3 \times 3 \times 3$

2 Lỗi 2: nghĩ mỗi RGB channel tạo output riêng

Không đúng.

Một filter xử lý toàn bộ R, G, B rồi cộng kết quả lại để tạo một feature map.

3 Lỗi 3: nghĩ 64 filter tạo 64 ảnh riêng biệt không liên quan

Chính xác hơn, chúng tạo 64 feature map và các feature map này tạo thành 64 channel của cùng một output tensor.

4 Lỗi 4: nhầm batch với channel

Tensor: `[32, 64, 16, 16]` có:

[32 , 64 , 16 , 16]

batch size = 32 | channel = 64 | height = 16 | width = 16

5 Lỗi 5: gọi Softmax trước CrossEntropyLoss

Không cần trong PyTorch. Đưa trực tiếp logits vào:

```python
nn.CrossEntropyLoss()
```

6 **Lỗi 6: tính sai đầu vào của Linear Layer**

Nếu tensor trước Flatten là $64 \times 8 \times 8$ thì Linear phải nhận 4096 feature.

XVII

PHẦN XVII
Mục 97–100 · 4 mục

Bài tập

Bốn bài tập có lời giải: tính output shape với stride 1 và stride 2, đếm tham số, theo dõi shape qua toàn bộ network.

97 Bài tập 1 — tính output shape

98 Bài tập 2 — tính output shape với stride 2

99 Bài tập 3 — tính số parameter

100 Bài tập 4 — theo dõi toàn bộ network

Mục 97 Bài tập 1 — tính output shape

Phần XVII

ĐỀ BÀI

Cho input $3 \times 64 \times 64$

Conv	kernel	padding	stride
$C_{\text{out}} = 32$	3×3	1	1

Hỏi output shape?

LỜI GIẢI

1. **Kích thước không gian** $H_{\text{out}} = \dfrac{64 + 2 - 3}{1} + 1 = 64 \qquad W_{\text{out}} = 64$
2. **Số output channel bằng số filter** 32
3. **Vậy** $32 \times 64 \times 64$

Mục 98 Bài tập 2 — tính output shape với stride 2

Phần XVII

ĐỀ BÀI

Input	kernel	padding	stride
64×64	3	1	2

LỜI GIẢI

1. **Ta có**

$$
\begin{aligned}
H_{\text{out}} &= \left\lfloor \frac{64 + 2 - 3}{2} \right\rfloor + 1 \\
&= \lfloor 31.5 \rfloor + 1 \\
&= 32
\end{aligned}
$$

2 **Vậy** 32×32

Mục 99 Bài tập 3 — tính số parameter

Phần XVII

ĐỀ BÀI

Conv:

`in_channels`	=	16	C_{in}
`out_channels`	=	32	C_{out}
`kernel_size`	=	3	K

LỜI GIẢI

1 **Số weight** $32 \times 16 \times 3 \times 3 = 4608$

2 **Bias** 32

3 **Tổng** 4640

Mục 100 Bài tập 4 — theo dõi toàn bộ network

Phần XVII

ĐỀ BÀI

Input $3 \times 64 \times 64$

Theo dõi shape qua Conv1 → Pool → Conv2 → Pool → Flatten và tìm số feature mà classifier đầu tiên phải nhận.

LỜI GIẢI

1 **Conv1** $3 \to 32$ `kernel 3` `padding 1` `stride 1`
Output $32 \times 64 \times 64$

2 **Pool** $32 \times 32 \times 32$

3 **Conv2** $32 \to 64$ `kernel 3` `padding 1` `stride 1`
Output $64 \times 32 \times 32$

4 **Pool** $64 \times 16 \times 16$

5 **Flatten** $64 \times 16 \times 16 = 16384$

Do đó classifier đầu tiên phải nhận `16384` feature.

XVIII

PHẦN XVIII
Mục 101–104 · 4 mục

Tổng kết

Toàn bộ CNN trong một chuỗi công thức, năm câu hỏi cho mỗi convolution layer, chuỗi tư duy từ pixel tới classification và kết luận.

101 Bản chất toàn bộ CNN trong một chuỗi công thức

102 Mô hình tư duy quan trọng nhất

103 Chuỗi tư duy từ pixel tới classification

104 Kết luận kiến thức cốt lõi

Mục 101 Bản chất toàn bộ CNN trong một chuỗi công thức

Phần XVIII

CNN có thể được nhìn dưới dạng:

INPUT	X
CONV	$Z^{(1)} = \mathrm{Conv}(X; W^{(1)}, b^{(1)})$
RELU	$A^{(1)} = \mathrm{ReLU}(Z^{(1)})$
POOL	$P^{(1)} = \mathrm{Pool}(A^{(1)})$
CONV	$Z^{(2)} = \mathrm{Conv}(P^{(1)}; W^{(2)}, b^{(2)})$
RELU	$A^{(2)} = \mathrm{ReLU}(Z^{(2)})$
POOL	$P^{(2)} = \mathrm{Pool}(A^{(2)})$
FLATTEN	$h = \mathrm{Flatten}(P^{(2)})$
LINEAR	$z = W_f h + b_f$
LOSS	$L = \mathrm{CrossEntropy}(z, y)$
BACKPROPAGATION	$\nabla_W L$
OPTIMIZER	$W \leftarrow W - \eta \nabla_W L$

☆ Toàn bộ CNN classification cơ bản có thể được hiểu thông qua chuỗi này.

Mục 102 Mô hình tư duy quan trọng nhất

Khi nhìn thấy một convolution layer, luôn đặt năm câu hỏi:

Câu 1 Input shape là gì?

$$C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}$$

Câu 2 Có bao nhiêu filter?

$$C_{\text{out}}$$

Câu 3 Mỗi filter có shape gì?

$$C_{\text{in}} \times K_H \times K_W$$

Câu 4 Spatial output là bao nhiêu?

$$H_{\text{out}}, W_{\text{out}}$$

Câu 5 Output tensor có shape gì?

$$C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$$

☆ Nếu trả lời được năm câu hỏi này, phần lớn sự nhầm lẫn về CNN sẽ biến mất.

Mục 103 Chuỗi tư duy từ pixel tới classification

CNN không phải một phép tính bí ẩn.

Toàn bộ quá trình có thể nhìn như:



☆ CNN học được các representation ngày càng phù hợp với nhiệm vụ.

Mục 104 **Kết luận kiến thức cốt lõi** Phần XVIII

CNN là Neural Network đặc biệt phù hợp với dữ liệu ảnh nhờ việc sử dụng các kernel nhỏ áp dụng lặp lại trên toàn bộ không gian ảnh.

Một convolution layer thực hiện về bản chất

multiply + sum + bias

tại từng vùng của input.

Với nhiều channel

sum qua width + sum qua height + sum qua channel

Với nhiều filter

mỗi filter → một output channel

Sau nhiều layer

low-level features → high-level features

Cuối cùng

features → logits → loss

và toàn bộ weight được học bằng

backpropagation + optimization

Ba kỹ năng quan trọng nhất cần thành thạo là:

1. Tính convolution bằng tay

ôn lại: Mục 7 · Mục 8 · Mục 21

2. Theo dõi tensor shape

ôn lại: Mục 56 · Mục 94

3. Hiểu quan hệ giữa $C_{\text{in}}, C_{\text{out}}, K, H, W$

ôn lại: Mục 25 · Mục 26

Khi ba phần này đã vững, người học có nền tảng để tiếp tục sang:

ResNet → Object Detection → Segmentation → Modern Vision Models

mà không còn xem CNN như một "hộp đen".

Convolutional Neural Network (CNN) · Công thức hiển thị bằng LaTeX (KaTeX).